



Universidad de Valladolid

Escuela Técnica Superior de Ingenieros de
Telecomunicación

Configuración y actualización dinámica de modelos
de aprendizaje profundo

GRADO EN INGENIERÍA DE TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO DE FIN DE GRADO

Autor:
Andrés García Treviño

Tutor:
Dr. D. Pablo Casaseca de la Higuera

Valladolid, junio 2026

Resumen

El crecimiento exponencial del número de dispositivos capaces de generar tráfico de red ha aumentado significativamente la necesidad de organizar los recursos disponibles para asegurar un nivel estable de calidad de servicio. La predicción del tráfico de red se ha convertido en la mayor aliada para evitar congestiones y pérdidas de servicio. Sin embargo, los modelos predictivos tradicionales tienen muchas dificultades para seguir el ritmo de cambio del tráfico de red actual. Como el tráfico de red puede ser modelado como una secuencia temporal, en este trabajo se estudian y comparan varios modelos de redes neuronales de aprendizaje profundo aplicadas a la predicción de series temporales con capacidad de adaptarse a las variaciones y anomalías mediante entrenamiento incremental. Se han estudiado varias arquitecturas que combinan redes convolucionales con redes recurrentes LSTM sobre el dataset Milan, que recoge el tráfico de red de la ciudad de Milán, Italia, entre noviembre y diciembre de 2013.

Los resultados de los modelos, analizados a través de diferentes métricas estadísticas, demuestran que los modelos que integran capas convolucionales y *online learning* son ampliamente superiores a los modelos tradicionales, incluso en situaciones de cambios bruscos en el patrón de los datos. El modelo ideal, presentando una capa CNN y una capa LSTM llega a reducir el error (NRMSE) cometido en las predicciones desde un 0.1778 conseguido con un modelo neuronal LSTM sencillo hasta un 0.142. Por último, los modelos analizados en este trabajo han sido optimizados al máximo para conseguir el mayor rendimiento posible en cuanto a carga computacional y tiempo de ejecución.

Palabras clave

Online learning, predicción, redes neuronales, redes convolucionales, tráfico de red.

Abstract

The exponential growth of telecommunication devices has raised the need of correctly allocating the network resources to ensure a reliable Quality of Service level. Network traffic prediction has become the greatest ally in the battle against network congestion and loss of service. Nonetheless the traditional prediction models are burdened with great difficulties to follow the changes in network traffic. As it can be seen as a temporal sequence, this work puts several *online deep learning* models under the microscope to elucidate whether they are able to adapt dynamically to changes in the temporal series. Various distinct architectures featuring convolutional and LSTM layers have been tested against the Milan dataset, which holds information about the traffic load in the city of Milan, Italy, between November and December 2013.

The models' outputs have been analysed via different statistical methods, demonstrating the vast superiority of the models combining convolutional and recurrent layers over the more traditional models, even when the pattern abruptly changes. The best model so far, characterized by a single CNN layer connected to an LSTM layer has been able to boost precision, measured as NRMSE, from 0.1778 to 0.142. Last but not least, all the models scrutinized by this work have been successfully optimized to achieve the greatest possible computation speeds.

Keywords

Convolutional networks, network traffic, neural networks, online learning, prediction.

Agradecimientos

A mi familia, a mis amigos, a mis profesores y, sobre todo, al Dr. D. Juan Pablo Casaseca de la Higuera. A todos ellos les agradezco su infinita paciencia.

Índice

1. Introducción	5
1.1. Objetivos	6
1.2. Estructura del documento	6
2. Fundamentos del <i>Machine Learning</i> y <i>Deep learning</i>	7
2.1. Inteligencia Artificial	7
2.2. <i>Machine Learning</i>	7
2.2.1. Técnicas de entrenamiento	8
2.2.2. <i>Online learning</i>	9
2.3. <i>Deep learning</i>	10
3. Redes neuronales	11
3.1. El <i>Perceptron</i>	11
3.2. Funciones de activación	12
3.3. Entrenamiento de redes neuronales y <i>backpropagation</i>	13
3.3.1. Gradient descent	13
3.3.2. Funciones de coste	14
3.3.3. <i>Backpropagation</i>	15
3.4. Selección de modelos	16
3.5. Técnicas de regularización	17
3.5.1. L_2 o <i>Weight Decay</i>	18
3.5.2. L_1 o <i>Lasso</i>	18
3.5.3. <i>Dropout</i>	19
3.5.4. <i>Early stopping</i>	19
3.6. Preprocesado de los datos de entrada	20
3.6.1. Entrenamiento en <i>mini-batch</i>	20

4. Predicción de series temporales	21
4.1. Modelos autorregresivos de medias móviles	22
4.2. Modelos de aprendizaje profundo aplicados a la predicción de series temporales	23
4.2.1. Redes neuronales recurrentes	24
4.2.2. Inconvenientes de las neuronas recurrentes	24
4.2.3. LSTM	25
5. Estructuras avanzadas de <i>Deep Learning</i> para la predicción de series temporales	27
5.1. CNNs	27
5.1.1. Mapas de filtros	28
5.1.2. Capas de agregación	28
5.2. CNN+LSTM	29
5.2.1. Neuronas, conexiones, ecuaciones, flujo de datos...	31
6. Resultados y discusión	32
6.1. Desempeño general	32
6.2. Análisis de Spearman	33
6.2.1. Número de celdas, capas CNN y tamaño del <i>kernel</i>	34
6.2.2. <code>look_back</code> y <code>batch_size</code>	35
6.2.3. <code>updates</code> y <code>threshold</code>	37
6.2.4. Número de filtros de las capas convolucionales	38
6.3. Mejor modelo	38
7. Conclusiones y líneas futuras	42
Referencias	43

Índice de figuras

1.	Distribución de las medias de NRMSE de los modelos con <i>online learning</i> según el tamaño de la cuadrícula. Elaboración propia.	32
2.	Distribución de las medias de NRMSE de los modelos con <i>online learning</i> según la arquitectura del modelo. Elaboración propia.	33
3.	Densidad de las medias de NRMSE en entrenamiento estático (línea discontinua) y <i>online learning</i> (línea continua) según el tamaño de la cuadrícula y del kernel. Elaboración propia.	35
4.	Influencia del hiperparámetro <code>look_back</code> en la media del NRMSE y en el tiempo de ejecución medio en modelos con <i>online learning</i> . Elaboración propia.	36
5.	Influencia del hiperparámetro <code>updates</code> en la media del NRMSE y en el tiempo de ejecución medio en modelos con <i>online learning</i> . Elaboración propia.	38
6.	NRMSE por celda del mejor modelo con <i>online learning</i> . Elaboración propia.	40
7.	NRMSE por celda de los mejores modelos con <i>online learning</i> de cada arquitectura (LSTM, 2CNN+LSTM y 3CNN+LSTM). Elaboración propia.	40

Índice de tablas

1. Coeficientes de Spearman (r_s) entre los hiperparámetros y las métricas de rendimiento como media del NRMSE y media del tiempo de ejecución en *online learning*. Elaboración propia. 34
2. Características del mejor modelo en *online learning* y sus métricas de rendimiento como media del NRMS de sus celdas. Elaboración propia. 39

1. Introducción

Las redes de telecomunicaciones tienen como finalidad el intercambio de información entre un nodo transmisor y un nodo receptor. Es un concepto sencillo afectado por un problema de números. Siguiendo la ley de Bell, el número de dispositivos ha aumentado exponencialmente desde 1 ordenador básico en un departamento de una empresa a múltiples dispositivos por persona (teléfonos móviles, relojes inteligentes...) [1]. Casi con la misma tendencia ha evolucionado la tecnología de las telecomunicaciones, impidiendo que el crecimiento de los dispositivos saturase las capacidades de la red. De esta forma, en la actualidad existe una infinidad de nodos transmisores y receptores.

Los flujos de información intercambiada entre los nodos transmisores y receptores forma el tráfico de red. Dicho tráfico puede ser representado y medido. Las medidas o muestras realizadas en diferentes instantes de tiempo pueden aunarse en una serie temporal. Como todas las series, cabe la posibilidad de que sigan un patrón que pueda utilizarse para predecir medidas futuras a partir del histórico de medidas pasadas [2]. Con la inmensa cantidad de flujos de información existentes a día de hoy, resulta imposible analizarlos mediante la intervención humana. Sin embargo, la situación planteada representa el caso de uso de la inteligencia artificial y *Deep Learning*: averiguar el patrón subyacente de un conjunto de datos [3].

La predicción del tráfico de red tiene numerosísimas aplicaciones como, por ejemplo, la correcta planificación y organización de los recursos de las redes de telecomunicaciones. Esto se traduce en un menor desperdicio de recursos, que a su vez se traduce en una inversión económica más eficiente, sin menoscabar la calidad del servicio proporcionado por la red de telecomunicaciones [4]. También es posible aplicar la predicción de tráfico de red a tareas como la detección de anomalías o intrusos, monitorización o mantenimiento [5].

En la actualidad existen multitud de aplicaciones que generan flujos de datos a demanda. Algunos ejemplos son el *media streaming* o los modelos de negocio *as a Service* (XaaS) [6]. Los modelos de predicción estáticos convencionales no son capaces de adaptarse al dinamismo y la falta de estacionariedad de dichos flujos de datos.

Para solucionar dicho problema se han desarrollado modelos capaces de adaptarse a los cambios en las tendencias o patrones subyacentes en los flujos de tráfico. Estos modelos se basan en *online active deep learning*. Esta técnica permite reentrenar los modelos a medida que se incorporan nuevas muestras para adaptarse a ellas en un proceso llamado *concept drift* [7–9].

Este trabajo se basa en la investigación realizada en [10,11] acerca de modelos LSTM que implementan actualizaciones incrementales donde las muestras sobre las que los modelos tuvieron un mayor error toman mayor importancia. En este trabajo se plantea la hipótesis de que un modelo que combine arquitecturas de redes neuronales convolucionales y de redes neuronales recurrentes tenga una precisión predictiva en *online learning* superior a aquellos modelos basados únicamente en redes neuronales recurrentes.

1.1. Objetivos

El principal objetivo de este trabajo es analizar la capacidad de adaptación mediante entrenamiento incremental de diferentes modelos de redes neuronales convolucionales recurrentes ante cambios en el patrón subyacente de series temporales.

Dentro del objetivo principal se establecen los siguientes objetivos secundarios:

- La comprensión y dominio del marco teórico utilizado en este trabajo.
- El manejo de herramientas de desarrollo como Python o Kaggle.
- La mejora del modelo base proporcionado.
- Evaluar el impacto producido por las redes neuronales convolucionales respecto al modelo base.
- Evaluar el rendimiento de los modelos en diferentes cuadrículas de 49 y 225 celdas.

1.2. Estructura del documento

Esta memoria se divide en 5 capítulos: introducción, antecedentes y estado del arte, entorno experimental, exposición de resultado y discusión y conclusiones y líneas futuras.

El Capítulo 1 se desarrolla la motivación y los objetivos de este trabajo. Además, se plantean la hipótesis y los objetivos y se añade una breve descripción de la estructura de la memoria.

El Capítulo ?? ofrece una visión general del marco teórico relevante a la predicción de series temporales, el aprendizaje automático, *deep learning*, *online learning* y las diferentes arquitecturas de redes neuronales convolucionales y recurrentes.

El Capítulo ?? presenta la base de datos utilizada, las arquitecturas analizadas y los criterios de selección y evaluación de modelos de aprendizaje automático aplicados en este trabajo.

El Capítulo 6 muestra y analiza el rendimiento de los distintos modelos propuestos. También interpreta los resultados obtenidos y comenta las relaciones presentes entre modelos y resultados.

El Capítulo 7 resume las aportaciones de este trabajo y presenta las conclusiones obtenidas. También propone líneas de actuación futuras.

2. Fundamentos del *Machine Learning* y *Deep learning*

2.1. Inteligencia Artificial

A lo largo de la historia el ser humano ha tratado de automatizar diversas tareas para obtener una mayor eficiencia en su ejecución, reducir su complejidad o liberar al ser humano de las mismas. La programación tal y como la conocemos ahora tiene el mismo objetivo y tuvo su inicio con Ada Lovelace y su *Analytical Engine* en 1843.

El *Analytical Engine* fue el primer ordenador de uso general conocido aunque se limitaba a repetir un mismo conjunto de órdenes perfectamente definidas. En 1950, un grupo de programadores da forma a lo que hoy conocemos como Inteligencia Artificial: la automatización de tareas que requieren del intelecto humano para ser realizadas y completadas.

Un gran ejemplo de Inteligencia Artificial fue DeepBlue: un programa que, a través del procesamiento de unos datos de entrada y una serie de reglas o patrones definidos que relacionan las entradas con las salidas, era capaz de jugar al ajedrez. DeepBlue no era capaz de realizar ningún tipo de aprendizaje y, aun así, fue una Inteligencia Artificial capaz de superar a grandes maestros mundiales del ajedrez como Gary Kasparov.

Aquellas inteligencias artificiales incapaces de aprender como DeepBlue son clasificadas como inteligencias artificiales simbólicas. Sin embargo, el concepto Inteligencia Artificial es muy amplio y engloba otros conceptos como el *Machine Learning*, que engloba a su vez el *Deep learning*.

2.2. *Machine Learning*

Deep Blue fue posible gracias a que sus programadores conocían a la perfección el patrón que relacionaba las entradas, en este caso la posición actual de las figuras de ajedrez en el tablero, y las salidas, siendo éstas últimas los movimientos que DeepBlue debía realizar sobre sus figuras en su turno de juego. Este patrón estaba extremadamente bien definido en las reglas del ajedrez y DeepBlue fue programado manualmente con un conjunto muy extenso de jugadas y decisiones que cubrían todas las posibilidades de juego.

Aun con la eficiencia mostrada por DeepBlue en el ajedrez, estos ejemplos de Inteligencia Artificial resultaron ser pésimos en la ejecución de tareas más complejas y poco definidas como la clasificación de imágenes o el procesamiento de texto.

Machine Learning, también llamado aprendizaje automático, le da una vuelta al problema: en vez de calcular las salidas mediante el procesamiento de las entradas y las reglas que relacionan las entradas y las salidas, se procesan las entradas y sus correspondientes salidas para averiguar los patrones que relacionan a ambas. De

esta forma, la Inteligencia Artificial puede embarcarse en la resolución de problemas cuyas reglas no están definidas de forma precisa o son demasiado extensas o arduas de definir.

Para estimar si los patrones calculados son correctos, es necesario evaluarlos midiendo la distancia entre las salidas reales o esperadas correspondientes a ciertas entradas y las salidas obtenidas aplicando las reglas obtenidas a las entradas. La forma en la que se calcula esta distancia está definida en la función de coste o *loss function*. Esta distancia es integrada de nuevo en el algoritmo de *Machine Learning* como retroalimentación para aumentar su precisión a la hora de averiguar las reglas entre las entradas y las salidas en iteraciones sucesivas. Este paso final completa el proceso de aprendizaje o entrenamiento, que finaliza cuando la distancia entre las salidas esperadas y las salidas obtenidas es suficientemente pequeña.

Si definimos los datos de entrada y sus salidas esperadas como E y a la función de coste como R , es posible definir el *Machine Learning* o aprendizaje automático: Un programa aprende de la experiencia E con respecto a una tarea T y una métrica de rendimiento R si su rendimiento en la tarea, medido por R , mejora con más experiencia E .

2.2.1. Técnicas de entrenamiento

Existen multitud de algoritmos de que pueden ser clasificados según su método de entrenamiento en: aprendizaje por refuerzo, aprendizaje no supervisado y aprendizaje supervisado.

El aprendizaje por refuerzo utiliza un sistema de recompensas y penalizaciones para que un agente de aprendizaje automático, mediante prueba y error, ajuste sus decisiones con el fin de maximizar las recompensas obtenidas y minimizar las penalizaciones aplicadas por el sistema.

Es especialmente útil para aprender decisiones secuenciales como el guiado autónomo de un robot por una ruta definida o la optimización de unos recursos como el uso de refrigeración industrial en centros de datos. En ambos casos, el sistema de recompensas asigna una valoración positiva a aquellas decisiones que mantienen al robot en la ruta o estabilizan la temperatura de los servidores mientras evalúa negativamente aquellas decisiones que desvían al robot o llevan la temperatura de los servidores fuera de un rango óptimo establecido.

Algunos ejemplos de algoritmos de aprendizaje por refuerzo son SARSA, Q-Learning...

El aprendizaje no supervisado está pensado para la organización y definición de un conjunto de datos. Los algoritmos englobados en esta técnica de entrenamiento deben encontrar los patrones y estructuras subyacentes en los datos de entrada, sin esperar una salida relacionada con los mismos. El aprendizaje no supervisado busca agrupar los datos en distintos sectores que maximicen la distancia intersector y minimicen la distancia intrasector.

Es especialmente útil en la clasificación de los datos según sus características como la segmentación de clientes, en la detección de anomalías como los fraudes financieros o en la reducción de la dimensionalidad de los datos como la simplificación de bases de datos. Algunos ejemplos de algoritmos de aprendizaje no supervisado son *k-means clustering*, *Support Vector Clustering* o el análisis de componentes principales (PCA).

Finalmente, el aprendizaje supervisado, en donde se enclaustra este trabajo, utiliza pares entrada-salida perfectamente definidos para entrenar sus algoritmos. Estos pares son conocidos como datos etiquetados y tienen la función de actuar como referencia para la medición de la distancia entre la salida obtenida y la salida esperada. Los modelos de aprendizaje supervisado deben, utilizando dichas distancias, ajustar de la forma más precisa posible el patrón que relaciona las entradas y las salidas.

Es especialmente útil en la clasificación como, por ejemplo, la identificación de correos electrónicos de SPAM y la predicción de datos como la predicción de eventos meteorológicos. Algunos ejemplos de algoritmos de aprendizaje supervisado son los árboles de decisión, *Support Vector Machines*, la regresión lineal y no lineal y las neuronas artificiales.

2.2.2. *Online learning*

Otra forma de clasificar los algoritmos de aprendizaje automático recae en su capacidad para aprender de forma incremental a medida que el modelo procesa nuevos datos. Los modelos estáticos u *offline* son entrenados con todos los datos disponibles y pierden la capacidad de modificarse a sí mismos una vez puestos en producción. Se convierten en algoritmos que aplican las mismas reglas a todos los nuevos datos que procesen sin excepción.

A medida que el tiempo avanza los datos disponibles cambian debido a la constante evolución de todo lo que nos rodea, y con ellos el patrón subyacente que se desea estimar a través de la inteligencia artificial. Este fenómeno, llamado *data drift* o *concept drift*, no afecta a los modelos estáticos por lo que se tornan cada vez más imprecisos e incapaces de proporcionar un resultado correcto. Por ejemplo, en el hipotético caso de la implementación de una nueva regla en la normativa del ajedrez, la soberanía de DeepBlue desaparecería. El mismo concepto es aplicable a los modelos de aprendizaje automático: un modelo estático de *clustering* entrenado sobre un mercado determinado perdería su eficacia ante un cambio en los patrones de compra o en las necesidades de los consumidores.

Para mitigar este problema, los modelos estáticos deben ser reentrenados de cero con todos los datos disponibles, incluidos los datos afectados por el *data drift* y volver a ser lanzados a producción. Esto requiere una gran cantidad de capacidad de cómputo y en tiempos de espera elevados mientras se reentrena el modelo, lo que se traduce en grandes inversiones económicas en recursos como CPUs, GPUs... y la pérdida de beneficios al no tener un buen modelo en producción durante el reentrenamiento.

Por otra parte, los modelos de aprendizaje incremental, también llamados modelos *online*, son entrenados en numerosas iteraciones sucesivas utilizando un conjunto reducido de datos en cada iteración. De esta forma cada iteración de entrenamiento es rápida y ligera. El modelo se adapta a los nuevos datos introducidos para evitar quedar obsoleto ante el *concept drift*. Esto implica que el proceso de aprendizaje no termina nunca, ni siquiera cuando el modelo es lanzado a producción.

Un aspecto importante del *online learning* es la tasa de aprendizaje. Dependiendo de la frecuencia con la que el patrón subyacente de los datos cambia, los modelos *online* deberán adaptarse en mayor o menor medida a los nuevos datos. Como los recursos *hardware* son limitados, también lo es la capacidad de los modelos. A medida que se adaptan a los nuevos patrones, pierden eficacia con los antiguos en un proceso llamado *catastrophic forgetting*.

2.3. *Deep learning*

Con todo esto, los algoritmos de *Machine Learning* son capaces de resolver problemas más complejos que la Inteligencia Artificial simbólica pero es posible ir un paso más allá. Aunque el *Machine Learning* pueda aprender los patrones que relacionan unas entradas con unas salidas, el *Deep learning*, también llamado aprendizaje profundo, utiliza varias capas jerárquicas de aprendizaje para combinar las estructuras más simples obtenidas en las primeras capas en patrones más complejos que representen los datos procesados de forma mucho más precisa.

En *Deep Learning*, los algoritmos utilizan capas de neuronas artificiales organizadas de forma sucesiva. Existe una gran variedad de neuronas artificiales, cada una de ellas especializada en una tarea concreta como aplicar un mismo filtro a un conjunto de datos de entrada, almacenar información relevante sobre los datos o reestructurar y redimensionar los datos procesados.

Al tener múltiples capas de neuronas, los modelos de aprendizaje profundo son capaces de procesar los datos sin ninguna intervención humana, mientras que los algoritmos de *Machine Learning* necesitan de la intervención del ser humano para refinar y adaptar los datos manualmente a las necesidades de cada algoritmo. Sin embargo, los modelos de *Deep Learning* necesitan grandes cantidades de datos para poder ser entrenados, lo que dificulta enormemente la interpretación de los procesos llevados a cabo por las redes neuronales, además de una gran inversión en componentes electrónicos especializados en dichos cálculos como las GPUs (graphical Processing Units). Un ejemplo de esta inversión se encuentra en el reciente giro radical de la producción de GPUs de Nvidia, el mayor fabricante de GPUs a nivel mundial, con el nuevo objetivo de satisfacer las ingentes necesidades del mercado del *Deep Learning*.

Aunque las redes neuronales nacieron en 1943 con el modelo lógico de neurona artificial de McCulloch y Pitts, no fueron rentables hasta el desarrollo de nuevas arquitecturas y funciones de activación, métodos de aprendizaje y ajuste como *back-propagation* y la mejora de los componentes electrónicos disponibles. En la actualidad, los modelos de aprendizaje profundo son los algoritmos predeterminados y

más eficientes en prácticamente todo el área de Inteligencia Artificial, desde visión artificial hasta procesamiento de lenguaje.

En estos últimos años desde la expansión del aprendizaje profundo, se han conseguido hitos como, entre otros, la clasificación de imágenes o el procesamiento de audio y lenguaje a nivel humano, la aparición de asistentes digitales y modelos conversacionales como ChatGPT o Microsoft Copilot o la mejora de sistemas de recomendaciones y búsquedas web.

Este trabajo se enmarca dentro del ámbito del *Deep Learning*, más concretamente en la predicción de tráfico de red a través de redes neuronales profundas. La predicción de series temporales, como es el caso del tráfico de red, es una tarea ardua que conlleva la estimación del patrón existente en la serie. Por lo tanto, cumple con todos los requisitos mencionados anteriormente para ser automatizada mediante algoritmos de aprendizaje profundo.

3. Redes neuronales

Las redes neuronales son los modelos de aprendizaje profundo por excelencia. Están formadas por neuronas artificiales que utilizan una serie de pesos para procesar unos datos de entrada. Su primera aparición fue en 1943 como una simple descripción del funcionamiento de las neuronas presentes en el sistema nervioso animal: una neurona con salida binaria activará su salida cuando un cierto número de sus entradas, también binarias, estén activas.

3.1. El *Perceptron*

Aunque las neuronas binarias pueden combinarse para formar redes capaces de procesar expresiones lógicas complejas, no son más que una combinación de puertas lógicas como las presentes en cualquier circuito electrónico. En 1957 Rosenblatt redefinió las neuronas artificiales para eliminar sus restricciones binarias, permitiendo un diseño basado en entradas y salidas continuas. Estas neuronas son llamadas TLU (*Threshold Linear Unit*).

Las TLUs reciben como entradas una serie de valores x_j donde J es la cantidad de entradas recibidas. Las TLUs calculan la combinación lineal z de las entradas con una serie de pesos w_j asociados a cada entrada y un término independiente b . Seguidamente, se aplica una función de activación $\phi(z)$ que acota la salida y de las TLUs en un rango de valores típicamente comprendido entre -1 y 1 o entre 0 y 1. De esta forma se evita que la salida de las TLUs crezca hasta infinito si se encadenan varias TLUs.

$$z = b + \sum_{j=1}^J w_j \cdot x_j = \mathbf{w}^\top \mathbf{x} + b \quad (1)$$

$$y = \phi(z) \quad (2)$$

Una neurona TLU con una función de activación binaria como la Heaviside puede ser utilizada en tareas como la clasificación binaria. Sin embargo, su verdadero poder reside en la agrupación de varias TLUs en una misma capa. Esta es la arquitectura de redes neuronales más básica: el *Perceptron*, capaz de, por ejemplo, clasificar los datos en $J+1$ categorías siendo J el número de neuronas TLU presentes en la arquitectura.

Cada TLU está conectada a todas las entradas en una disposición llamada capa completamente conectada o *dense layer*. Esta disposición no está presente en todas las arquitecturas como, por ejemplo, en las neuronas convolucionales. La Figura ?? describe un *Perceptron* de dos entradas y tres TLUs con tres salidas. En este caso, en forma matricial con tantas filas i como TLUs y tantas columnas j como entradas, los pesos $w_{i,j}$ se agrupan en la matriz $\mathbf{W}$ y las entradas y los términos independientes en los vectores $\mathbf{x}$ y $\mathbf{b}$ respectivamente, modificando las expresiones 1 y 2:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \quad (3)$$

$$\mathbf{y} = \phi(\mathbf{z}) \quad (4)$$

Los *Perceptrons* son en sí arquitecturas muy simples con capacidades limitadas pero pueden encadenarse para aumentar su capacidad de cómputo. Esta arquitectura se llama *Perceptron* multicapa o MLP (*Multilayer Perceptron*). Así, las salidas de la primera capa de neuronas TLU es utilizada como entrada para la segunda capa de TLUs de forma sucesiva, como se observa en la Figura ??.

Denotando como $\mathbf{h}_l$ la salida de la capa l en un MLP con L capas y a los pesos de las neuronas de la misma capa como $\mathbf{W}_l$ y $\mathbf{b}_l$, se modifican las ecuaciones 3 y 4 de la forma:

$$\mathbf{h}_l = \phi(\mathbf{W}^\top \mathbf{h}_{l-1} + \mathbf{b}_l) \quad (5)$$

$$\mathbf{y} = \mathbf{h}_L \quad (6)$$

Los MLPs pueden ser utilizados en diversas tareas como la clasificación o la regresión. Son capaces incluso de predecir varios valores al mismo tiempo dependiendo del número de TLUs presentes en la última capa de neuronas.

3.2. Funciones de activación

Existe una gran multitud de funciones de activación, todas ellas no lineales, para cada tarea en la que se apliquen las neuronas artificiales. Su implementación per-

mite que las neuronas realicen transformaciones no lineales de los datos, siendo así capaces de ajustarse a una mayor variedad de patrones. Sin embargo, las funciones más básicas como la Heaviside no son compatibles con los métodos actuales de entrenamiento de modelos de aprendizaje profundo. Por ello, se propusieron otras funciones más complejas como la función sigmoide $\sigma(z)$, la función linear rectificada $\text{ReLU}(z)$ (*Rectified Linear Unit*) y la tangente hiperbólica $\tanh(z)$.

Las funciones de activación han sufrido un empujón en los últimos años para eliminar las áreas de las funciones como ReLU con gradientes nulos o puntos no derivables. Algunas de ellas son Leaky ReLU y Mish.

3.3. Entrenamiento de redes neuronales y *backpropagation*

Como se ha mencionado anteriormente, además de neuronas y funciones de activación, es necesario dotar al modelo de aprendizaje profundo de una métrica que evalúe la precisión de sus resultados y un método matemático capaz de modificar la red neuronal basándose en dicha métrica. Para obtener salidas distintas, la forma más sencilla es modificar ligeramente los valores contenidos en la matriz de pesos $\mathbf{W}$ de forma aleatoria. Sin embargo, existen protocolos más eficaces como *gradient descent*.

3.3.1. Gradient descent

Gradient descent es un algoritmo de entrenamiento que modifica los parámetros que influyen en la salida de un modelo con el objetivo de minimizar una función de coste. Los parámetros de una red neuronal son los pesos $\mathbf{W}$ incluyendo los términos independientes $\mathbf{B}$. La variación introducida en los parámetros está controlada por la tasa de aprendizaje o *learning rate* η .

Con cada cambio en los parámetros, es posible calcular cuánto varía la función de coste a través de su gradiente con respecto a los parámetros modificados $\nabla_{\mathbf{W}}$. Un gradiente positivo indica la dirección en la que la función de coste aumenta, por lo que los cambios deberán ir en la dirección opuesta. De esta forma, denominando LF a la función de coste utilizada, el algoritmo *gradient descent* se resume en:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (7)$$

donde $\mathbf{W}^{t-1}$ son los parámetros iniciales y $\mathbf{W}^t$ son los parámetros resultantes.

A lo largo de múltiples iteraciones, *gradient descent* es capaz de obtener los pesos adecuados para cada neurona de tal forma que minimicen la función de coste aplicada. Sin embargo, las funciones de coste presentan valles que podrían confundir al algoritmo *gradient descent* haciéndole creer que se encuentra en el mínimo absoluto. Por ello, es común añadir un término aleatorio α que permita escapar de los mínimos locales. Añadiéndolo a la Ecuación 7 se consigue la expresión para *stochastic*

gradient descent:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \alpha \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (8)$$

Estas variaciones en la expresión matemática que define el algoritmo *gradient descent* son los llamados optimizadores. Existen una gran cantidad de ellos como SDG (*Stochastic Gradient Descent*), *Momentum*, RMSProp o Adam. Cada uno de ellos implementa una mejora de la eficiencia respecto al anterior.

En este trabajo se utiliza un optimizador para aumentar la eficiencia de *gradient descent* propuesto en [10]. Partiendo de la premisa de que las iteraciones de *gradient descent* que producen un error mayor en la función de coste ofrecen más información que aquellas con errores más pequeños, dar un peso mayor a las primeras aumentará la eficacia del algoritmo. Para ello se modifica la Ecuación 7 añadiendo la norma euclídea al cuadrado del error cometido en cada iteración:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \left\| \frac{y - \hat{y}_{t-1}}{y} \right\|^2 \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (9)$$

donde y es la salida real esperada e $\hat{y}_{t-1}$ es la salida arrojada por la red neuronal en la iteración $t - 1$ de *gradient descent*.

3.3.2. Funciones de coste

Las funciones de coste son una parte esencial del algoritmo *gradient descent* y no están relacionadas con las funciones de activación. Son el objetivo a minimizar a través de la modificación de los parámetros de la red neuronal. El cambio en los parámetros modifica el procesamiento que realiza la red sobre los datos de entrada y, consecuentemente, en la salida obtenida. La función de coste debe ajustarse al objetivo de la red neuronal para que los resultados sean válidos.

El algoritmo *gradient descent* permite a la red neuronal tomar cualquier atajo para minimizar su función de coste como ignorar ciertos datos clave o inventar nuevos datos. Por ello, ante la gran diversidad de funciones de coste existente a día de hoy, no todas las funciones son implementables para todas las redes neuronales ni para todos los propósitos.

Entre las funciones de coste más populares se encuentran: la entropía cruzada binaria para tareas de clasificación binaria, su variante categórica para clasificación multiclase, similitud del coseno para procesamiento de textos o el error cuadrático medio (MSE) para regresión y procesamiento de series temporales. En este trabajo se utilizará la función de coste MSE (*Mean Squared Error*).

3.3.3. *Backpropagation*

El algoritmo *gradient descent* es la base del entrenamiento de redes neuronales pero no es aplicable directamente a redes con múltiples capas de neuronas. En 1970 Seppo Linnainmaa introdujo el algoritmo de diferenciación automática invertida (*reverse-mode automatic differentiation*) que calcula en dos pasadas a toda la red todos los gradientes del error cometido en la salida, medido por la función de coste, respecto a todos y cada uno de los parámetros de la red neuronal. De esta forma se obtiene la contribución al error de cada término independiente $b_{i,l}$ y cada peso $w_{i,j,l}$ de cada entrada j de cada neurona i de cada capa l . El algoritmo *gradient descent* puede utilizar estas aportaciones al error para calcular la magnitud en la que cada parámetro debe ser modificado.

El algoritmo *backpropagation*, también llamado retropropagación, es la combinación del algoritmo de Linnainmaa con *gradient descent*. En primera instancia, la red neuronal procesa los datos de entrada para calcular una salida. Este es el pase hacia delante o *forward pass*. Es el funcionamiento normal de una red neuronal salvo que se guardan todas las salidas de todas las neuronas de todas las celdas en vez de desecharlas y dar únicamente la salida final. Con todas esas salidas se ejecuta *reverse-mode automatic differentiation* desde la última capa hasta la primera para calcular la contribución al error perteneciente a cada capa. Este es el pase hacia atrás o *backward pass*. Por último, todos los parámetros son modificados con *gradient descent*.

Es muy importante que los parámetros estén inicializados de forma aleatoria para que se puedan distinguir los unos de los otros. Si no fuera así, el algoritmo *reverse-mode automatic differentiation* sería incapaz de saber qué neurona ha contribuido más o menos al error y *gradient descent* las actualizaría en la misma cuantía. Al ser todas las neuronas iguales, actuarían como una sola con la consecuente reducción de la capacidad de cómputo de la red neuronal.

Por motivos similares no todas las funciones de activación son válidas. Aunque la función de coste indica el error final cometido en la salida, *backpropagation* debe calcular la contribución de cada parámetro al total. Cambiar un peso $w_{i,j,l}$ o un término independiente cualquiera $b_{i,l}$ de cualquier neurona no supone un cambio directo en la salida de dicha neurona sino que ha de pasar por la función de activación. Por lo tanto, la función de activación debe ser diferenciable para que *backpropagation* pueda calcular cuánto ha de modificar el peso o término término independiente deseado para mover la salida final en una dirección determinada.

Por esta razón, todas las funciones no diferenciables utilizan trucos para sustituir sus puntos no diferenciables con otros valores. Sin embargo, las funciones de activación más problemáticas son aquellas con gradientes nulos puesto que anulan el efecto de *gradient descent*. Un gran ejemplo de gradientes nulos es la función Heaviside(z), que tiene gradiente cero para todo z salvo $z = 0$ donde no es diferenciable.

La función de activación utilizada en este trabajo y en la gran mayoría de redes neuronales es ReLU(z). Tiene un gradiente no nulo para $z > 0$, un punto no diferenciable en $z = 0$ y gradiente nulo para $z < 0$. Esto implica que, si la combinación

de entradas y pesos de la neurona es menor a cero, la neurona queda inutilizada. Es una gran ventaja que permite a la red neuronal eliminar neuronas innecesarias para cada dato de entrada.

La elección de ReLU como función de activación no sólo recae en su capacidad para generar gradientes no nulos para $z > 0$. Los componentes electrónicos sobre los que se realizan las operaciones típicas de las redes neuronales tienen limitaciones. Funciones de activación muy complejas requieren grandes inversiones en capacidad de cómputo. Además, a nivel de *software* y librerías de programación, la función ReLU está altamente optimizada para obtener el máximo rendimiento de la electrónica.

Las redes neuronales basadas en *online learning* ejecutan el algoritmo *backpropagation* en cada entrenamiento incremental. La precisión de las salidas es evaluada constantemente y los pesos de las capas de neuronas son modificados consecuentemente para minimizar la función de coste.

3.4. Selección de modelos

Los modelos de redes neuronales son entrenados aplicando *backpropagation* hasta que los pesos de la red son los idóneos para minimizar la función de coste. Sin embargo, los pesos no son los únicos parámetros que caracterizan a la red neuronal evaluada. Hay una gran multitud de variables, llamadas hiperparámetros, que influyen en los resultados obtenidos. Un conjunto de hiperparámetros con unos valores concretos identifica a un modelo.

Existen diferentes estrategias en la búsqueda de los mejores hiperparámetros. Entre las estrategias más comunes se encuentran el *grid search* y el *random search*. La primera itera sobre una matriz de valores concretos establecidos mientras la segunda utiliza conjuntos de valores obtenidos de forma aleatoria dentro de un rango delimitado. *Grid search* ofrece un gran control sobre los valores utilizados en la búsqueda mientras *random search* ofrece la posibilidad de encontrar buenos conjuntos de valores en un rango de búsqueda más amplio. Ninguna estrategia es infalible, siempre cabe la posibilidad de que el modelo para el que se realiza la búsqueda no sea un buen modelo [12].

Según cómo se diseñe el programa o modelo de aprendizaje automático y el valor de sus hiperparámetros, su rendimiento a la hora de obtener el patrón subyacente de unos datos puede variar mucho. No existe *a priori* un modelo mejor que otro [13]. Por ello, se ha diseñado un protocolo de selección de modelos de aprendizaje automático.

El aprendizaje automático se divide en tres fases que contienen 3 sets de datos diferentes: entrenamiento, validación y test. El entrenamiento refiere a la primera fase, donde el programa o modelo aprende en la medida que le es posible el patrón subyacente presente en los datos de entrenamiento a través del algoritmo *backpropagation*. El grado de aprendizaje correcto del patrón de los datos de entrenamiento se denomina error de entrenamiento o E_{in} , siendo 0 el mejor valor posible.

El modelo, una vez pasada la fase de entrenamiento, dejará de aprender para con-

vertirse en un modelo en producción que se dedicará a aplicar el patrón aprendido en el entrenamiento a nuevas muestras. Sin embargo, aunque el modelo tenga un E_{in} bajo, puede no dar las respuestas correctas a las nuevas muestras. El error cometido en las muestras en producción se denomina como E_{out} y el error de generalización se calcula como $E_{out} - E_{in}$. Para evitar que esto ocurra y se lance un modelo a producción que no funcione bien, provocando en la mayoría de casos pérdidas económicas y oportunidades de negocio, se introducen las fases de validación y test.

No se pueden comparar modelos entre sí directamente usando E_{in} puesto que esto sólo conllevaría a la elección del modelo con menor E_{in} y, en consecuencia, con mayor E_{out} . Esta situación es conocida como *overfitting*. La fase de validación se encarga de comparar los diferentes modelos propuestos haciéndoles procesar un set de datos de validación como si estuvieran en producción. El desempeño de los modelos en esta fase o E_{val} sí que puede utilizarse para comparar modelos.

Aunque existen múltiples estrategias de validación como *Grid Search*, *Random Search*, *k-fold*... no todas son aplicables a la predicción de series temporales. Los datos de validación siempre han de ser posteriores a los datos de entrenamiento y los datos de test siempre han de ser posteriores a los datos de validación [14]. Por lo tanto, estrategias como *k-fold validation* quedan descartadas de este trabajo.

La fase de validación tiene el mismo problema que la fase de entrenamiento. Cabe la posibilidad de que el modelo escogido haya realizado *overfitting* en los datos de validación y, aunque tenga un valor pequeño de E_{val} , tenga un gran error E_{out} . Por ello, la fase de test reentrena únicamente el mejor modelo de la fase de validación utilizando los datos de entrenamiento y validación y mide el nuevo rendimiento con un set de datos de test. El error E_{test} es entonces una buena estimación de E_{out} .

Es importante recalcar que los datos de test sólo deben ser utilizados en la fase de test. Utilizarlos en otras fases como la de entrenamiento o en el reentrenamiento del modelo tras la validación provocará una distorsión en la medida E_{test} llamada *data leakage*. Tampoco ha de usarse para comparar modelos como si fuera un test de validación [3, 15, 16].

3.5. Técnicas de regularización

Los problemas derivados del *overfitting* tienen un gran impacto en el modelo escogido puesto que éste debe tener un buen rendimiento al procesar datos ajenos a los datos de entrenamiento, validación y test. Un modelo con *overfitting* sólo es capaz de estimar el patrón subyacente en los datos de entrenamiento. Por ello, además de la estrategia de validación en la selección de modelos, existe la regularización.

Un modelo con una gran cantidad de parámetros o de neuronas con respecto a la complejidad de los datos procesados es capaz de aprender de las muestras de entrenamiento pero también del ruido presente en los mismos. Esto implica que el modelo estima un patrón subyacente que presenta fluctuaciones aleatorias propias del ruido, siendo mucho menos eficaz. La regularización es un conjunto de técnicas cuyo objetivo es aplicar una penalización a la complejidad del modelo para evitar el

overfitting.

Para evitar que una bajada en E_{in} provoque una subida inaceptable de E_{out} y, en consecuencia, del error de generalización, existen varias técnicas de regularización:

3.5.1. L_2 o *Weight Decay*

Previene que el algoritmo *gradient descent* o *backpropagation* resulten en valores desmedidos de los pesos $\mathbf{W}$. Los pesos grandes permiten que cualquier fluctuación aleatoria o pasajera en el patrón de los datos sea amplificadas en la red neuronal.

La función de coste $LF(W^{t-1})$ presente en la ecuación 7 introduce un término de penalización en forma de norma euclídea al cuadrado de la magnitud de los pesos y un término λ para controlar el impacto de la regularización.

$$LF(\mathbf{W}^{t-1})_{L_2} = LF(\mathbf{W}^{t-1}) + \frac{\lambda}{2} \|\mathbf{W}^{t-1}\|_2^2 = LF(\mathbf{W}^{t-1}) + \frac{\lambda}{2} \sum_{l=1}^L \sum_{i=1}^{I_l} \sum_{j=1}^{J_l} w_{i,j,l}^2 \quad (10)$$

siendo L la cantidad total de capas de neuronas, I_l la cantidad total de entradas de la capa l y J_l la cantidad total de neuronas de la capa l .

La ecuación 7 queda de la siguiente forma:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1})_{L_2} = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) - \eta \lambda \mathbf{W}^{t-1} \quad (11)$$

$$\mathbf{W}^t = (1 - \eta \lambda) \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) \quad (12)$$

El término $(1 - \eta \lambda) \mathbf{W}^{t-1}$ contrae continuamente los pesos en cada iteración del algoritmo *backpropagation* logrando reducir el impacto del *overfitting*. El término λ permite aumentar o reducir el efecto de la regularización para evitar que el modelo pase al escenario contrario o *underfitting*.

3.5.2. L_1 o *Lasso*

Es otra técnica de regularización que actúa sobre la función de coste $LF(\mathbf{W}^{t-1})$ de una forma muy parecida a L_2 . Sin embargo, su objetivo es la eliminación de las conexiones o neuronas redundantes o con poco impacto en la salida.

Utilizando el valor absoluto de los pesos como mecanismo de penalización se consigue reducir la magnitud de los mismos en una cantidad constante. También se añade un término λ para controlar el impacto de la regularización.

$$LF(\mathbf{W}^{t-1})_{L_1} = LF(\mathbf{W}^{t-1}) + \lambda \|\mathbf{W}^{t-1}\|_1 = LF(\mathbf{W}^{t-1}) + \lambda \sum_{l=1}^L \sum_{i=1}^{I_l} \sum_{j=1}^{J_l} |w_{i,j,l}| \quad (13)$$

El operador ∇ transforma la norma de los pesos en la función signo, modificando la ecuación 7 de la forma:

$$\mathbf{W}^t = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1})_{L_1} = \mathbf{W}^{t-1} - \eta \nabla_{\mathbf{W}} LF(\mathbf{W}^{t-1}) - \eta \lambda \text{sgn}(\mathbf{W}^{t-1}) \quad (14)$$

El operador $-\eta \lambda \text{sgn}(\mathbf{W}^{t-1})$ empuja los pesos hacia cero, anulando aquellos demasiado pequeños como para ser considerados útiles.

3.5.3. *Dropout*

Propuesta por Geoffrey Hinton en 2012, es una de las técnicas más populares de regularización. El objetivo reside en forzar a las neuronas a tener en cuenta todas las conexiones, no sólo una porción de ellas. Como se ha comentado anteriormente, la red neuronal puede tomar cualquier atajo para minimizar la función de coste, incluyendo la coadaptación de neuronas.

La coadaptación de neuronas es un fenómeno en el que ciertas neuronas se ajustan para corregir los errores que otras generaron en el set de entrenamiento. Esto genera dependencias entre neuronas que sólo son útiles durante el entrenamiento, reduciendo E_{in} y aumentando E_{out} .

Dropout es un algoritmo simple que consiste en apagar neuronas, es decir, truncar su salida a cero, con una probabilidad p en cada *mini-batch* de entrenamiento. Apagar aleatoriamente algunas neuronas fuerza a la red a romper las coadaptaciones y reforzar conexiones que de otra manera serían ignoradas. Las neuronas son forzadas a ser independientes a la vez que cooperativas.

3.5.4. *Early stopping*

Durante el proceso de entrenamiento los modelos aprenden gradualmente a estimar el patrón subyacente presente en los datos de entrenamiento guiándose por la función de coste. Sin embargo, anular completamente la función de coste reduciría tanto E_{in} que el error de generalización sería demasiado alto.

Por ello, un mecanismo muy simple de regularización consiste en detener el proceso de entrenamiento cuando nuevos datos de entrenamiento reduzcan E_{in} pero no reduzcan E_{val} en una cuantía predeterminada. Esta técnica se denomina *early stopping*.

3.6. Preprocesado de los datos de entrada

Aunque la elección de un buen modelo con los hiperparámetros correctos es muy importante, también lo es la disponibilidad de datos veraces. Los datos utilizados tanto para el entrenamiento como la validación o el test han de ser: suficientemente numerosos como para permitir al algoritmo *backpropagation* llegar al mínimo de la función de coste y representativos del patrón subyacente que se desea estimar.

Las características de los datos de entrada son altamente influyentes en el rendimiento de los modelos de aprendizaje automático. Por ello se ha de actuar sobre los datos para evitar errores en su procesado. Las acciones más comunes son: imputación y normalización. La imputación se encarga de los posibles huecos en los datos, rellenándolos con ceros u otro valor significativo para no interrumpir el patrón subyacente a estimar. La normalización aplica una transformación a los datos para desplazarlos y escalarlos en el rango $[0, 1]$.

La normalización es especialmente útil para evitar que los modelos ignoren los datos más pequeños en favor de los datos más grandes. Sin embargo, la normalización se aplica respecto a los datos incluidos únicamente en el set de entrenamiento. El resto de los datos, tanto del set de validación como de test, son transformados utilizando los valores mínimo min_{train} y máximo max_{train} del set de entrenamiento:

$$x_{norm} = \frac{x - min_{train}}{max_{train} - min_{train}} \quad (15)$$

De cualquier otra forma, el modelo tendría acceso a los datos del conjunto de validación o test en la fase de entrenamiento. El modelo se adaptaría erróneamente a unos datos no representativos en un proceso llamado *data leakage*.

3.6.1. Entrenamiento en *mini-batch*

Uno de los grandes problemas de los modelos de aprendizaje profundo es la necesidad de grandes volúmenes de datos para formar sets de entrenamiento, validación y test. Por ello, la práctica estándar implementa entrenamiento por *mini-batch*. En vez de procesar todos los datos del conjunto de entrenamiento para realizar *backpropagation*, se procesa un conjunto reducido distinto de dichas muestras cada vez. Por ejemplo, un modelo de *deep learning* enfocado a la identificación de ciertos objetos en una imagen procesaría una cantidad reducida de imágenes de entrenamiento, estimaría las salidas correspondientes, las compararía con la realidad y ajustaría sus pesos con *backpropagation*. Este proceso se repetiría con distintos sets de imágenes de entrenamiento hasta procesar el set entero.

Este TFG tiene como objetivo procesar los datos relativos al tráfico de red presente en la ciudad de Milán. Dichos datos forman una serie temporal continua cuya longitud es excesivamente grande. Procesar toda la serie cada vez que se realiza *backpropagation* requeriría una elevada inversión en tiempo de procesamiento y en *hardware* capaz de procesar tantos datos al mismo tiempo. Además, en la fase de

online learning se debe alimentar a los modelos neuronales con distintas secuencias de datos para obtener una cantidad suficiente de predicciones que permitan evaluar los modelos. Por ello es imperativo segmentar la serie temporal en secuencias más cortas.

Para la segmentación de series temporales se implementa una técnica de ventana deslizante que crea múltiples secuencias más cortas a partir de la serie original. Aunque este TFG trabaje con múltiples series temporales al mismo tiempo, una por cada celda geográfica de la ciudad, se puede modelar como una única serie temporal multivariable. Dadas D series que representan el tráfico de red a lo largo de T instantes cronológicos $\mathbf{X} \in \mathbb{R}^{T \times D}$, se crean $T - LB$ secuencias de LB (*lookback*) muestras formuladas como:

$$\mathbf{X}_k = [\mathbf{x}_k, \mathbf{x}_{k+1}, \mathbf{x}_{k+2}, \dots, \mathbf{x}_{k+LB-1}]^\top \quad (16)$$

$$\mathbf{Y}_k = \mathbf{x}_{k+LB} \quad (17)$$

Como se observa en las ecuaciones 16 y 17, las series temporales son fácilmente divisibles para las tareas de predicción ya que tras realizar la segmentación, la salida esperada para cada secuencia $\mathbf{X}_k$ es la muestra $\mathbf{x}_{k+LB}$.

Los modelos de *deep learning* se benefician enormemente de esta estrategia al no tener que procesar cantidades desmesuradas de datos para ajustar sus pesos $\mathbf{W}$ sino un conjunto de B secuencias cortas $X_k, \dots, X_k + B$ que conforman un *mini-batch*.

4. Predicción de series temporales

Una serie temporal puede ser representada como un vector en cuyas posiciones se encuentran muestras secuenciales y equiespaciadas en el tiempo de una misma variable. Existen multitud de métodos de predicción de series de datos basados en la extrapolación. Sin embargo, no son específicos de series temporales sino que se aplican a conjuntos de datos genéricos.

El estudio clásico de las series temporales considera que su comportamiento es fruto de la tendencia, las variaciones cíclicas y estacionales y un componente aleatorio. La tendencia es la componente que refleja la evolución lineal a largo plazo de la serie temporal de forma lineal, exponencial... La componente cíclica recoge las oscilaciones periódicas de amplitud superior a un año. Por último, la componente estacional recoge las oscilaciones periódicas de amplitud inferiores a un año como las oscilaciones mensuales, semanales, diarias... Estas tendencias pueden generar una combinación aditiva si son independientes entre sí o multiplicativa si están correlacionadas entre sí [17].

En la actualidad es mucho más común recurrir a modelos de aprendizaje automático para las tareas de predicción. La mayoría de estos modelos se basan en arquitecturas neuronales recurrentes, capaces de memorizar los aspectos temporales más

importantes de la serie analizada.

4.1. Modelos autorregresivos de medias móviles

Como su propio nombre indica, los modelos autorregresivos de medias móviles para la predicción de series temporales están basados en los modelos de medias móviles y autorregresivos. Las medias móviles son una manera simple de obtener una predicción cuando no hay una tendencia marcada ni tampoco estacionalidad. Se utiliza el término medias móviles puesto que conforme se va disponiendo de nuevas observaciones se pueden calcular nuevas medias mediante la inclusión de las nuevas muestras y el desprendimiento de las más antiguas [18]. De esta forma se eliminan las oscilaciones de mayor frecuencia, actuando como un filtro paso bajo.

Las medias móviles unilaterales permiten estimar el valor futuro de la serie temporal $\hat{y}_t$ como la media de las m muestras inmediatamente anteriores:

$$\hat{y}_t = \frac{1}{m} \cdot \sum_{i=1}^m y_{t-i} \quad (18)$$

Asímismo, la media móvil puede combinarse con una ponderación k_i . La ponderación más habitual es la exponencial asumiendo que las muestras más recientes contribuyen más a la correcta predicción que las muestras más antiguas:

$$\hat{y}_t = \frac{1}{m} \cdot \sum_{i=1}^m y_{t-i} \cdot k_{t-i} \quad (19)$$

El modelo ARMA (*AutoRegressive Moving Average*) es el modelo más sencillo que combina las medias móviles y la autorregresión. Utiliza tanto las muestras reales de la serie temporal Y_t como los errores cometidos en predicciones pasadas ϵ_t . Concretamente, la cantidad de muestras y errores cometidos que son tomados en cuenta está estipulada por los parámetros p y q respectivamente. Siendo α_t y θ_t la ponderación aplicada a cada muestra y error [15, 19, 20]:

$$\hat{y}_t = \sum_{i=1}^p \alpha_i \cdot y_{t-i} + \sum_{i=1}^q \theta_i \cdot \epsilon_{t-i} \quad (20)$$

$$\epsilon_t = y_t - \hat{y}_t \quad (21)$$

El primer sumatorio es la parte autorregresiva, realiza una regresión basada en datos pasados. El segundo sumatorio es la media móvil de los errores cometidos en predicciones pasadas. Este modelo se denomina ARMA (p,q).

ARMA fue desarrollada inicialmente para la predicción de datos financieros [21] pero ha dado grandes resultados en sus adaptaciones a la predicción de tráfico en

red a corto plazo [22, 23]. Sin embargo, el modelo ARMA presupone que la serie temporal es estacionaria, es decir, que sus propiedades estadísticas son constantes en el tiempo, sin tendencias ni comportamientos periódicos [15].

El modelo ARIMA (*AutoRegressive Integrated Moving Average*) [24] introduce la distancia entre dos muestras como instrumento para eliminar las tendencias presentes en la serie temporal. Al diferenciar el valor de una muestra al valor de la siguiente se reduce un orden de la tendencia. Aplicar una segunda diferenciación se reduce otro orden más, etc. De esta forma, la serie temporal resultante tras d órdenes de diferenciación es estacionaria y se puede aplicar el modelo ARMA. A este proceso se le denomina ARIMA(p, d, q).

A su vez, el modelo ARIMA falla en analizar las oscilaciones periódicas que puedan estar presentes en los datos. El modelo SARIMA (*seasonal ARIMA*) mejora el modelo ARIMA añadiendo un procesamiento en paralelo de la componente periódica para una frecuencia determinada. Este procesamiento adicional también es realizado a partir de un modelo ARIMA con sus propios parámetros P, D, Q imitando los parámetros p, d, q . Así, la notación de este último modelo es SARIMA(p, d, q)(P, D, Q, s) siendo s el periodo de la componente frecuencial analizada.

4.2. Modelos de aprendizaje profundo aplicados a la predicción de series temporales

En los últimos años, los modelos autorregresivos descritos anteriormente han sido relevados por modelos de aprendizaje automático y, más concretamente, aprendizaje profundo. Éstos últimos son especialmente populares gracias a su capacidad de extraer el patrón subyacente de los datos sin necesidad de intervención humana para adaptar los algoritmos a las series temporales. El *deep learning* elimina la necesidad de analizar los datos en busca de características estacionarias, tendencias, componentes periódicas, anomalías... Un mismo modelo de aprendizaje profundo es capaz de adaptarse por sí mismo a cualquier serie temporal, lo que no disminuye la importancia de las estrategias de entrenamiento y selección de modelos como la normalización de los datos o la validación.

Las redes neuronales han evolucionado desde su creación para mejorar su rendimiento en los diferentes escenarios en los que se aplican, aunque no son exclusivas de ellos. Por ejemplo, el *Perceptron* dio lugar a las neuronas convolucionales para tareas de reconocimiento visual. En el caso de las series temporales, las redes neuronales evolucionaron hacia las neuronas recurrentes que forman las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*).

Las RNNs son utilizadas en todo tipo de secuencias temporales de cualquier naturaleza como muestras de audio, registros históricos numéricos e incluso documentos escritos. Para realizar sus predicciones, al igual que los modelos clásicos autorregresivos, las RNNs y todos los modelos de aprendizaje automático asumen en sus predicciones que el patrón subyacente presente en los datos de entrenamiento se perpetúa en los datos futuros.

4.2.1. Redes neuronales recurrentes

Las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*), a diferencia de las TLUs, no están basadas en ninguna estructura biológica en concreto. Son capas de neuronas capaces de retener información de muestras procesadas con anterioridad. En cada momento t , las neuronas recurrentes más simples, además de procesar las entradas $\mathbf{x}_t$ actuales, procesan su propia salida anterior $\hat{y}_{t-1}$ con su propio peso $w_{\hat{y}}$ para generar la salida actual $\hat{y}_t$. Esto hace a las RNN ideales para el procesamiento de secuencias de datos como, por ejemplo, el histórico de tráfico de telecomunicaciones de una ciudad concreta.

$$\hat{y}_t = \phi(\mathbf{W}_x^\top \mathbf{x}_t + b + w_{\hat{y}} \cdot \hat{y}_{t-1}) \quad (22)$$

Al igual que las TLUs, las neuronas recurrentes pueden agruparse para formar capas. Modificando la ecuación 22 con la forma matricial de los pesos $\mathbf{W}_x$, $\mathbf{W}_{\hat{y}}$ y la forma vectorial de las salidas $\hat{\mathbf{y}}_t$, la salida de una capa de neuronas recurrentes se describe como:

$$\hat{\mathbf{y}}_t = \phi(\mathbf{W}_x^\top \mathbf{x}_t + \mathbf{W}_{\hat{y}}^\top \hat{\mathbf{y}}_{t-1} + \mathbf{b}) \quad (23)$$

donde ϕ sigue siendo la función de activación presente en la neurona. Gracias al algoritmo *gradient descent*, las capas de neuronas recurrentes son capaces de adaptarse al patrón subyacente de las series temporales sin necesidad de intervención humana.

Es importante recalcar la naturaleza de $\mathbf{W}_x$, $\mathbf{W}_{\hat{y}}$ y $\mathbf{b}$. A diferencia de los vectores de entradas y salidas, los pesos de las neuronas recurrentes son los mismos independientemente del momento t , al igual que los pesos de las TLUs y los MLPs. Esto permite tratar a una misma secuencia como un sólo dato, manteniendo la integridad de las capas de neuronas a lo largo de toda la serie temporal.

La similitud con las TLUs y los *Perceptrons* continúa al poder agrupar varias capas de neuronas recurrentes de forma secuencial como un MLP. Las salidas $\hat{\mathbf{y}}_t$ de las capas inferiores conforman las entradas $\mathbf{x}_t$ de las capas superiores, pudiendo aplicar el algoritmo *backpropagation* para encontrar los valores idóneos de $\mathbf{W}_x$, $\mathbf{W}_{\hat{y}}$ y $\mathbf{b}$.

La retroalimentación de la salida $\hat{y}$ en una neurona recurrente es el mecanismo que otorga la capacidad de memoria, así como lo hacía la autorregresión en los modelos clásicos. Las RNNs procesan secuencias de datos mientras mantienen la información de las muestras pasadas en el llamado estado oculto h_t . Este estado puede ser directamente la salida pasada $h_t = \hat{y}_{t-1}$ o una transformación más compleja de la misma.

4.2.2. Inconvenientes de las neuronas recurrentes

Al igual que las TLUs, las neuronas recurrentes son estructuras muy simples por lo que su rendimiento está acotado por el ámbito en el que se utilizan. Concretamente,

las RNNs sufren varios problemas al enfrentar secuencias de larga duración.

El primer problema recae en la inestabilidad de la retroalimentación. Como los pesos $\mathbf{W}_x$, $\mathbf{W}_{\hat{y}}$ y $\mathbf{b}$ son constantes a lo largo de toda la serie temporal, se aplican tantas veces como instantes temporales tenga la secuencia procesada. La retroalimentación puede provocar que las salidas disminuyan hasta anularse o, con una función de activación no saturante como ReLU, aumenten sin control hasta inutilizar toda la red.

Algunos mecanismos para evitar la inestabilidad incluyen funciones de activación que saturan la salida como la tangente hiperbólica, una tasa de aprendizaje η más pequeña o *gradient clipping*.

El segundo problema reside en, debido a la simpleza de las neuronas recurrentes, su limitada capacidad de memoria. A medida que la red neuronal procesa secuencias $\mathbf{x}_t$, los pesos $\mathbf{W}_{\hat{y}}$ modifican el estado h_t eliminando la información más antigua en favor de la información más reciente.

Esto supone un gran problema puesto que la información más reciente no siempre es útil para estimar el patrón subyacente de la serie temporal. Sin un mecanismo de decisión, el estado oculto h_t es extremadamente vulnerable a cualquier anomalía o dato irrelevante presente en la secuencia procesada.

4.2.3. LSTM

Para resolver las limitaciones descritas anteriormente se propuso la neurona LSTM (*Long-Short Term Memory*) en 1997. La primera diferencia radica en la duplicación del estado $\mathbf{h}_t$ en dos vectores de estado $\mathbf{c}_t$ y $\mathbf{h}_t$. Uno de los vectores, $\mathbf{c}_t$, contiene la información que la neurona LSTM considera digna de ser recordada a largo plazo y el otro, $\mathbf{h}_t$ es el encargado de la salida de la neurona.

Las neuronas LSTM no son una neurona simple sino una estructura modular que contiene varias puertas y operaciones lógicas para controlar el flujo interno de información. Cada puerta es en realidad un *Perceptron* con pesos entrenables cuya función de activación es la sigmoide $\sigma(\mathbf{z})$. Todas las puertas combinan las entradas $\mathbf{x}_t$ con el vector de corto alcance $\mathbf{h}_{t-1}$ para orquestar el circuito de información.

Las puertas controlan la información existente en los vectores de estado. El vector de largo plazo $\mathbf{c}_t$ sólo es visto por la neurona LSTM. Crea un bucle de retroalimentación que actúa como la memoria de la neurona. El vector de corto plazo $\mathbf{h}_t$ pasa por las puertas y, aun siendo retroalimentado también conforma la salida de la neurona hacia la siguiente capa.

- **Puerta de olvido $\mathbf{f}_t$:** También llamada *forget gate*, combina las entradas x_t y el vector a corto plazo $\mathbf{h}_{t-1}$ para decidir qué parte del vector $\mathbf{c}_{t-1}$ ya no es útil a la vista de la nueva entrada.

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf}^\top \mathbf{x}_t + \mathbf{W}_{hf}^\top \mathbf{h}_{t-1} + \mathbf{b}_f) \quad (24)$$

siendo $\mathbf{W}_{xf}$ los pesos de la puerta de olvido aplicados a la entrada, $\mathbf{W}_{hf}$ los pesos aplicados al vector de corto plazo y $\mathbf{b}_f$ el término independiente.

Al utilizar la función sigmoide se acota la salida $\mathbf{f}_t$ entre 0 y 1. Este valor indica la porción del vector $\mathbf{c}_{t-1}$ que permanecerá intacto. Esto ocurre gracias a una operación de multiplicación elemento a elemento:

$$\mathbf{f}_t \odot \mathbf{c}_{t-1} \quad (25)$$

- **Capa de entrada $\mathbf{g}_t$:** Aunque no es estrictamente una puerta de control, es la capa principal que combina las entradas y el vector de corto alcance. Contiene el mecanismo de una neurona recurrente simple:

$$\mathbf{g}_t = \tanh(\mathbf{W}_{xg}^\top \mathbf{x}_t + \mathbf{W}_{hg}^\top \mathbf{h}_{t-1} + \mathbf{b}_g) \quad (26)$$

siendo sus elementos análogos a los de la ecuación 24. El término $\mathbf{g}_t$ es el candidato a estado de largo alcance basado en la nueva muestra $\mathbf{x}_t$.

- **Puerta de entrada $\mathbf{i}_t$:** También llamada *input gate*, controla la porción del estado candidato $\mathbf{g}_t$ que será añadido al estado de largo alcance tras aplicar la puerta de olvido. De forma análoga a la puerta de olvido:

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi}^\top \mathbf{x}_t + \mathbf{W}_{hi}^\top \mathbf{h}_{t-1} + \mathbf{b}_i) \quad (27)$$

Junto a la puerta de olvido, la puerta de entrada controla las actualizaciones del vector de largo alcance de la forma:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{g}_t \odot \mathbf{i}_t \quad (28)$$

- **Puerta de salida $\mathbf{o}_t$:** También llamada *output gate*, decide qué porción del vector de largo alcance actualizado $\mathbf{c}_t$ será utilizada tanto como salida hacia la siguiente capa de neuronas $\mathbf{y}_t$ como hacia la retroalimentación $\mathbf{h}_t$. De forma análoga a la puerta de olvido:

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo}^\top \mathbf{x}_t + \mathbf{W}_{ho}^\top \mathbf{h}_{t-1} + \mathbf{b}_o) \quad (29)$$

$$\hat{\mathbf{y}}_t = \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (30)$$

Cada puerta tiene su set de pesos independiente del resto de puertas, lo que permite entrenarlos para la tarea, ya sea olvido, entrada o salida, a la que son aplicados. Ésto consigue realizar una selección automática de la información útil para ser memorizada y utilizada a lo largo de la serie temporal.

Las neuronas LSTM no tienen un método especial de entrenamiento, utilizan *back-propagation* como el resto de neuronas. Por lo tanto, también pueden generar *overfitting* si la capacidad de procesamiento de la red neuronal es superior a la complejidad de la serie temporal. En ese caso el vector de largo alcance incluiría las fluctuaciones aleatorias propias del ruido de los datos de entrada.

5. Estructuras avanzadas de *Deep Learning* para la predicción de series temporales

5.1. CNNs

Las redes neuronales convolucionales o CNNs (*Convolutional Neural Networks*) están compuestas por neuronas parecidas a las TLUs. Sin embargo, las neuronas convolucionales no están conectadas a todas las entradas sino a un cierto conjunto próximo a ellas.

Las neuronas convolucionales fueron propuestas en 1980 imitando la estructura del córtex visual animal descrita en 1959. En dichos estudios base se descubrió la existencia de campos receptivos locales en las neuronas visuales animales que se activan selectivamente ante ciertos estímulos concretos. Por ejemplo, aunque dos neuronas compartan un mismo campo visual, una sólo reacciona a patrones verticales y la otra a patrones horizontales.

Estas neuronas resultan ideales para tareas de visión artificial o reconocimiento de imágenes puesto que los elementos espacialmente más próximos tienden a exhibir una correlación mayor que los elementos lejanos. Los campos receptivos de las neuronas visuales animales están adaptadas para procesar únicamente los elementos cercanos, consiguiendo una mayor eficiencia y precisión. Las neuronas artificiales convolucionales siguen la misma premisa, siendo especialmente útiles al procesar datos organizados en dos dimensiones espaciales.

Utilizando como ejemplo el caso de estudio de este TFG, la ciudad de Milán es un plano bidimensional dividido en múltiples celdas. Es plausible pensar que una determinada celda está más correlacionada con una celda adyacente que con una celda lejana. Las neuronas convolucionales atienden únicamente a las celdas próximas, ignorando las celdas lejanas. Utilizando varias neuronas es posible cubrir toda la ciudad.

Aunque las celdas lejanas estén menos correlacionadas, pueden presentar el mismo patrón subyacente. Por ello las neuronas convolucionales aplican un mismo filtro a todas las celdas ejecutando así una convolución a lo largo de todos los datos de entrada.

Entre las principales ventajas de las neuronas convolucionales se encuentra la reducción del número de parámetros de la red neuronal. Cada conexión de una neurona con una entrada necesita un peso w entrenable. Las neuronas TLU están conectadas a todas las entradas, necesitando un gran volumen de pesos, lo que conlleva una gran inversión en capacidad de procesamiento.

Aun así, al entrar en el ámbito del *deep learning*, las neuronas convolucionales presentan un problema: al tener campos receptivos locales, las capas de la red neuronal son cada vez más pequeñas. Además las neuronas situadas en los extremos de las capas tienen un campo visual más reducido. La solución es sencilla: un perímetro de ceros o *zero padding* alrededor del plano de los datos permite que las capas

mantengan su tamaño sin afectar a la salida. Tomando como referencia la ecuación 3:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} + 0 = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \quad (31)$$

5.1.1. Mapas de filtros

Como se ha mencionado anteriormente, las neuronas visuales animales reaccionan ante estímulos concretos. Las neuronas artificiales convolucionales se agrupan en mapas de filtros. Cada filtro o *kernel* es un conjunto de pesos aplicado a las conexiones de las neuronas para que reaccionen a estímulos diferentes. El *kernel* $\mathbf{W}_f$ es compartido por todas las neuronas que forman el mapa del filtro f .

Las neuronas convolucionales se organizan en cuadrículas bidimensionales o mapas, lo que permite definir sus campos visuales con dos variables espaciales que se denominarán M para la altura y N para la anchura en este TFG. Siendo u y v la posición espacial de una neurona convolucional en el mapa de filtros f de la capa l , ésta está conectada a las neuronas en las filas $[u, u + M - 1]$ y a las columnas $[v, v + N - 1]$ de todos los mapas de filtros f' de la capa inmediatamente anterior $l - 1$.

$$z_{u,v,f,l} = b_{f,l} + \sum_{f'=0}^{F'-1} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} w_{m,n,f',l} \cdot x_{u+m,v+n,f',l} \quad (32)$$

$$x_{u+m,v+n,f',l} = \hat{y}_{u+m,v+n,f',l-1} \quad (33)$$

Los *kernels* son entrenables mediante *backpropagation*. Aplicar filtros sobre regiones pequeñas resulta muy eficiente para la detección de bordes o cambios en el espacio. Al compartir un mismo *kernel* entre todas las neuronas de un mapa, se consigue realizar la convolución entre el filtro y toda la superficie de los datos.

Los campos visuales de dos neuronas adyacentes no tienen por qué estar desfasados en una celda exactamente, pueden estar desfasados tantas celdas como indique el *stride*. En cada paso de la convolución, el filtro se desplaza un número de celdas determinado por el *stride*, existiendo *stride* horizontal y vertical que no tienen por qué ser idénticos. Esto provoca que, aun con la presencia del *zero padding*, las capas superiores sean más pequeñas que las inferiores cuando el *stride* es superior a uno.

Encadenar múltiples capas de neuronas convolucionales permite que las capas iniciales detecten patrones sencillos mientras las capas superiores evalúan la presencia de patrones más complejos a partir de los detectados en las capas inferiores.

5.1.2. Capas de agregación

Al mantener constante el tamaño de las capas, las redes CNN profundas, aun teniendo menos conexiones que los MLPs, acumulan un gran volumen de parámetros

entrenables. Durante el proceso de retropropagación el modelo ha de guardar todas las salidas de todos los filtros de todas las capas para poder calcular el gradiente del error. Esto requiere una gran inversión en memoria y capacidad de cómputo.

Para solucionar este problema se implementaron las capas de agregación o *pooling*. Las neuronas de las capas de *pooling* no contienen *kernels* ni ningún parámetro entrenable. Simplemente realizan una operación matemática sobre las salidas de las neuronas contenidas en su campo visual. Dicha operación matemática está orientada a reducir la complejidad de la red, fusionando varias entradas en una sola con funciones como la media o el máximo del campo visual.

Las neuronas de *pooling* son especialmente útiles para introducir cierto grado de invarianza traslacional en la red equivalente a realizar un submuestreo de los datos. Gracias a ello la red es capaz de identificar la presencia de un patrón subyacente independientemente de dónde esté situado espacialmente. Por ejemplo, un modelo de *deep learning* basado en CNNs cuya misión es detectar la presencia de cierto objeto en una imagen no necesita ser capaz de delimitar a la perfección el contorno del objeto en la imagen sino de detectar su presencia y su posición de forma aproximada.

En la práctica es común intercalar capas de *pooling* con capas de neuronas convolucionales estándar. Sin embargo, en el contexto de las series temporales, la introducción forzada de invarianza traslacional mediante capas de agregación no suele ser útil, cuando no contraproducente. Aunque en el reconocimiento de imágenes del ejemplo anterior sea muy útil, en la predicción de series temporales la posición temporal exacta es imprescindible. En el caso de este TFG el momento exacto en el que se produce un pico de tráfico de red es muy importante y las capas de agregación distorsionarían esa información. Además, las secuencias temporales de las celdas en las que se divide la ciudad se difuminarían entre sí, perdiendo la identidad de cada celda.

En otros contextos de series temporales sí serían útiles, como por ejemplo la detección de anomalías sin necesidad de identificar el momento exacto del *outlier*. Todas estas tareas están mucho más relacionadas con la clasificación que con la predicción.

5.2. CNN+LSTM

En este TFG se proponen varias arquitecturas basadas en la combinación de capas convolucionales y recurrentes. La idea parte de la naturaleza espaciotemporal de los datos de tráfico de red de la ciudad de Milán. La ciudad está formulada como un plano dividido en celdas. Cada celda contiene en un instante dado la cantidad de tráfico de red cursada. Se propone entonces utilizar las capacidades de análisis espacial de las neuronas convolucionales para estimar patrones geográficos en cada instante y las capacidades de análisis temporal para analizar la evolución de los patrones geográficos en el tiempo. Esta arquitectura se denomina comúnmente como *ConvLSTM* o *Time Distributed CNN*.

El plano de la ciudad estudiado en este TFG puede modelarse como una matriz cuadrada de tamaño $M_{total} \times N_{total}$ de entradas a la red neuronal $\mathbf{X}_t$ en cada instante

de tiempo. Esta matriz es entregada a las capas convolucionales para la extracción de los patrones geográficos $\mathbf{G}_t$ presentes en cada instante. Cada patrón contiene F mapas de filtros.

$$\mathbf{G}_t = \text{ReLU}(\mathbf{W}_{cnn}^\top \mathbf{X}_t + b_{cnn}) \in \mathbb{R}^{M_{total} \times N_{total} \times F} \quad (34)$$

Las capas convolucionales están seguidas de capas recurrentes LSTM por lo que para realizar una tarea de predicción se deben agrupar varios patrones G_t en secuencias de longitud LB aptas para su procesamiento por las neuronas LSTM.

Los modelos neuronales propuestos, tanto en la modalidad *offline* como *online*, utilizan entrenamiento por *mini-batches*. Cada *mini-batch* agrupa una cantidad BS (*batch-size*) de secuencias de patrones $\mathbf{G}_t$.

$$\mathbf{B}_k = \begin{bmatrix} \mathbf{G}_k & \mathbf{G}_{k+1} & \cdots & \mathbf{G}_{k+LB-1} \\ \mathbf{G}_{k+1} & \mathbf{G}_{k+2} & \cdots & \mathbf{G}_{k+LB} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{G}_{k+BS-1} & \mathbf{G}_{k+BS} & \cdots & \mathbf{G}_{k+BS+LB-2} \end{bmatrix} \in \mathbb{R}^{BS \times LB \times M_{total} \times N_{total} \times F} \quad (35)$$

Los *mini-batches* $\mathbf{B}_k$ ¹ se relacionan entre sí por contener BS secuencias temporales de longitud LB en forma de ventana deslizante hasta completar todo el set de datos disponible. Las secuencias están formadas a su vez por los patrones geográficos extraídos en cada instante.

Las series temporales resultan sencillas de manipular en contextos de *mini-batches* puesto que la salida esperada $\mathbf{Y}_k$ para cada uno es la siguiente muestra a la última del *mini-batch*, facilitando la medición del error en la función de coste. Continuando con la ecuación 35:

$$\mathbf{Y}_k = \mathbf{X}_{k+BS+LB-1} \quad (36)$$

Sin embargo, los *batches* B_k son improcesables por las neuronas LSTM sin proyectarlos en una matriz tridimensional. El proceso es sencillo y simplemente fusiona las tres últimas dimensiones de B_k en una sola para obtener $B_k \in \mathbb{R}^{BS \times LB \times (M_{total} \cdot N_{total} \cdot F)}$. No se produce ningún cambio en los datos ni pérdidas de información. Es una simple reorganización de los elementos de una matriz.

Por último, los *batches* tridimensionales son procesados por las capas recurrentes para extraer el patrón temporal que define la evolución de los patrones geográficos y generar una predicción $\hat{\mathbf{Y}}_k$. En este TFG se utilizan exclusivamente neuronas LSTM en las capas recurrentes por su superioridad respecto a las neuronas recurrentes simples.

¹Se utiliza la numeración k en vez de t puesto que los *mini-batches* no representan una secuencia temporal sino un conjunto de datos de entrenamiento para una red neuronal.

$$\hat{\mathbf{Y}}_k = LSTM(\mathbf{B}_k) \tag{37}$$

6. Resultados y discusión

6.1. Desempeño general

El objetivo de este trabajo es la evaluación del rendimiento de los modelos con *online learning* respecto a los mismos modelos con entrenamiento estático u *offline learning*. En todas las combinaciones de hiperparámetros y en todas las arquitecturas los modelos con *online learning* han mostrado un rendimiento superior. Por lo tanto, en este capítulo se analizarán principalmente los resultados obtenidos en los modelos con *online learning*.

El *grid search* realizado en este trabajo no es uniforme. Para ahorrar tiempo se ha comenzado con un *grid search* reducido que se ha ido expandiendo según la correlación entre los hiperparámetros y sus resultados. Por ello, es difícil aislar la influencia de cada hiperparámetro en los resultados aunque analizando todos en conjunto es posible obtener conclusiones coherentes.

Como se ha mencionado anteriormente, la comparación entre los diferentes modelos en el proceso de validación se realiza con la media aritmética del NRMSE obtenido en cada celda. Como en toda media aritmética, la cantidad de valores de la variable a medir influye en gran medida en el resultado final. Como es de esperar, las medias del NRMSE de los modelos con una cuadrícula de 15x15 celdas tienen una desviación menor que las medias del NRMSE de los modelos con una cuadrícula de 7x7 celdas, como se observa en la Figura 1. Esto es debido a que la variación en el NRMSE de una celda es amortiguada por las otras 224 celdas, mientras que 48 celdas tienen un poder amortiguador menor.

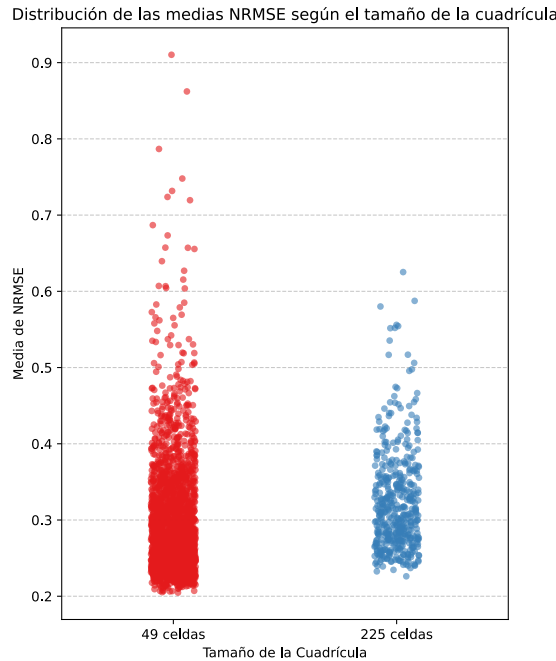


Figura 1: Distribución de las medias de NRMSE de los modelos con *online learning* según el tamaño de la cuadrícula. Elaboración propia.

Según la arquitectura, es decir, el número de capas convolucionales utilizadas, la distribución de las medias de NRMSE es más o menos constante. Como se observa en los diagramas de violín de la Figura 2, al aumentar el número de capas convolucionales, el rendimiento de los modelos se concentra más en torno a 0.27 de media de NRMSE aunque tienen más *outliers*.

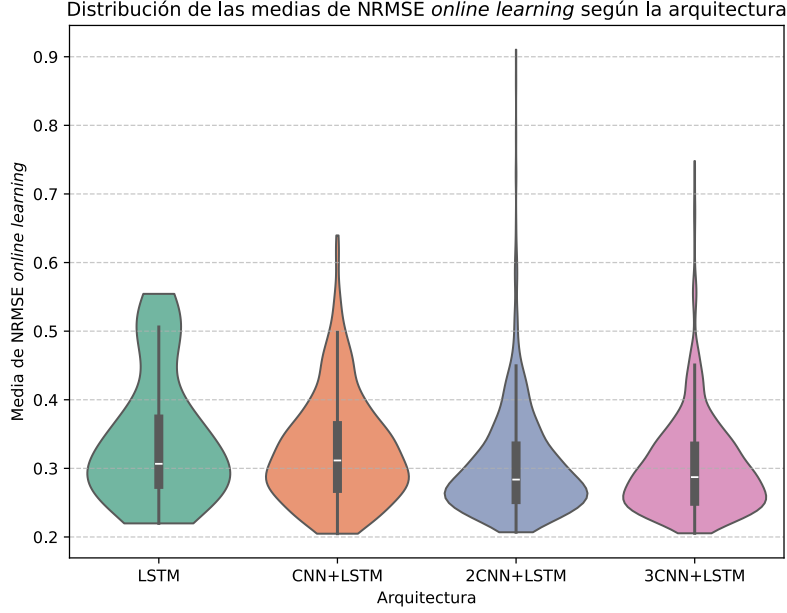


Figura 2: Distribución de las medias de NRMSE de los modelos con *online learning* según la arquitectura del modelo. Elaboración propia.

Aun así, aunque las arquitecturas con más capas y, por ende, más capacidad de procesamiento sean menos susceptibles a cambios en los hiperparámetros, la arquitectura con una única capa convolucional (CNN+LSTM) es la que ha conseguido el mejor modelo.

6.2. Análisis de Spearman

Tras un vistazo rápido a los resultados según el tamaño del dataset y de la arquitectura utilizada, el análisis de Spearman puede arrojar más luz acerca de la contribución de los hiperparámetros a los resultados obtenidos por los modelos con *online learning*. La Tabla 1 muestra el coeficiente de Spearman (r_s) para todos los hiperparámetros y métricas de rendimiento.

	NRMSE train	NRMSE test <i>offline</i>	NRMSE test <i>online</i>	Tiempo (s) <i>online</i>
Tamaño cuadrícula	0,4715	0,6598	0,1613	0,3127
Capas CNN	-0,3044	0,2384	-0,0980	0,5366
Tamaño <i>kernel</i>	-0,2282	-0,3540	-0,0271	-0,1942
<i>look_back</i>	-0,2321	0,3183	-0,2465	0,5053
<i>batch_size</i>	-0,0055	-0,0087	-0,1993	0,1711
<i>updates</i>	-0,1094	0,0101	-0,3473	0,5482
<i>threshold</i>	0,0343	-0,2324	0,0362	-0,2177
Filtros capa 1	0,0068	-0,0815	0,0574	-0,2144
Filtros capa 2	0,0121	0,0238	0,1108	-0,1523
Filtros capa 3	-0,0286	0,0479	0,0659	-0,1166

Tabla 1: Coeficientes de Spearman (r_s) entre los hiperparámetros y las métricas de rendimiento como media del NRMSE y media del tiempo de ejecución en *online learning*. Elaboración propia.

6.2.1. Número de celdas, capas CNN y tamaño del *kernel*

El primer hiperparámetro es el tamaño de la cuadrícula, que está muy relacionado de forma directa con el error cometido en el entrenamiento estático ($r_s = 0,6598$). A medida que se procesan más celdas al mismo tiempo, la red tiene mayores dificultades para generalizar. Sin embargo, el tamaño de la cuadrícula afecta mucho menos al rendimiento de la red cuando se implementa *online learning* ($r_s = 0,1613$). Es un gran indicador de la capacidad de los modelos de adaptarse de forma dinámica a los cambios en los patrones de los datos independientemente de la cantidad de datos procesados.

Algo similar ocurre con el número de capas convolucionales de la red neuronal. A medida que se añaden más capas los modelos *online* mejoran, aunque no demasiado ($r_s = -0,098$). Al contrario de lo que pudiera pensarse, aumentar la capacidad de procesamiento de la red mediante la adición de capas CNN no es beneficioso. Una mayor capacidad de la red provoca *overfitting*, tal y como indica el coeficiente de Spearman entre el número de capas convolucionales y el error cometido por los modelos estáticos ($r_s = 0,2384$).

El siguiente hiperparámetro es el tamaño del *kernel*, es decir, el tamaño del campo de visión de las neuronas convolucionales y está estrechamente relacionado con el tamaño de la cuadrícula. La arquitectura formada únicamente por una capa de neuronas recurrentes LSTM no se ve afectada por este hiperparámetro. Un campo de visión grande en una cuadrícula pequeña suele ser contraproducente puesto que no permite realizar muchas convoluciones. A su vez, un *kernel* pequeño en una cuadrícula grande puede no ser capaz de captar patrones extendidos sobre áreas más extensas.

Para este trabajo se ha incluido en el *grid search* los datasets de 49 y 225 celdas y *kernels* de tamaño 2x2 y 3x3, enfocando la mayor parte del *grid* en el dataset de 49 celdas con un *kernel* 3x3. Un *kernel* mayor de 4x4 sería demasiado grande y no permitiría a las capas convolucionales obtener detalles del patrón de los datos. Obviamente, un *kernel* de 1x1 carece de sentido.

En la Figura 3 se puede observar la influencia tanto del tamaño de la cuadrícula como del campo de visión de las neuronas convolucionales. La influencia del tamaño del *kernel* no es tan grande como la del tamaño de la cuadrícula pero en todos los escenarios un *kernel* menor es ligeramente más eficiente aunque no es una mejora suficientemente grande como para ser tomada en cuenta. Además, su coeficiente de Spearman en entrenamiento estático ($r_s = -0,354$) es menor al del número de celdas y en *online learning* es completamente despreciable ($r_s = -0,0271$).

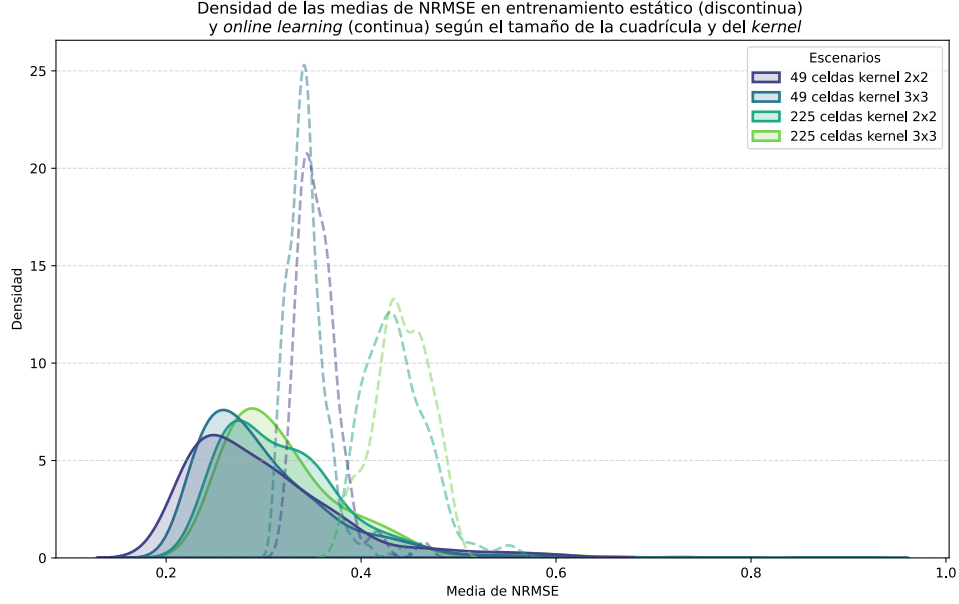


Figura 3: Densidad de las medias de NRMSE en entrenamiento estático (línea discontinua) y *online learning* (línea continua) según el tamaño de la cuadrícula y del *kernel*. Elaboración propia.

6.2.2. look_back y batch_size

Los hiperparámetros *look_back* y *batch_size* son los que mayor correlación tienen con la media de NRMSE de los modelos con *online learning* junto al hiperparámetro *updates*. Ambos hiperparámetros afecta a los modelos en todas sus fases de entrenamiento y validación. El parámetro *look_back* indica la longitud en horas de las secuencias temporales que la red neuronal procesa y *batch_size* indica la cantidad de esas secuencias que se procesan al mismo tiempo.

Los datos utilizados presentan una componente periódica marcada de forma diaria y semanal. Por lo tanto, es razonable ajustar el parámetro *look_back* a dicha frecuencia. En el presente trabajo se ha probado con valores entre 12 y 32 horas con la intención de permitir que la red aprenda la componente diaria. La componente semanal queda fuera de consideración puesto que un valor grande del hiperparámetro *look_back* impondría una carga computacional excesiva, pudiendo aumentar demasiado el tiempo de procesamiento de los datos.

La tendencia de la media de NRMSE en los modelos con *online learning* respecto al valor de *look_back*, tal y como indica su coeficiente de Spearman ($r_s = -0,2465$),

es inversa. Aun así, su coeficiente de Spearman es muy alto con respecto al tiempo de ejecución ($r_s = 0,5053$). Como se observa en la Figura 4, aunque aumentar la longitud de las series temporales analizadas lleve a una mayor precisión, el tiempo de ejecución aumenta demasiado. Los mejores valores para el hiperparámetro `look_back` están entre 18 y 24 horas de longitud.

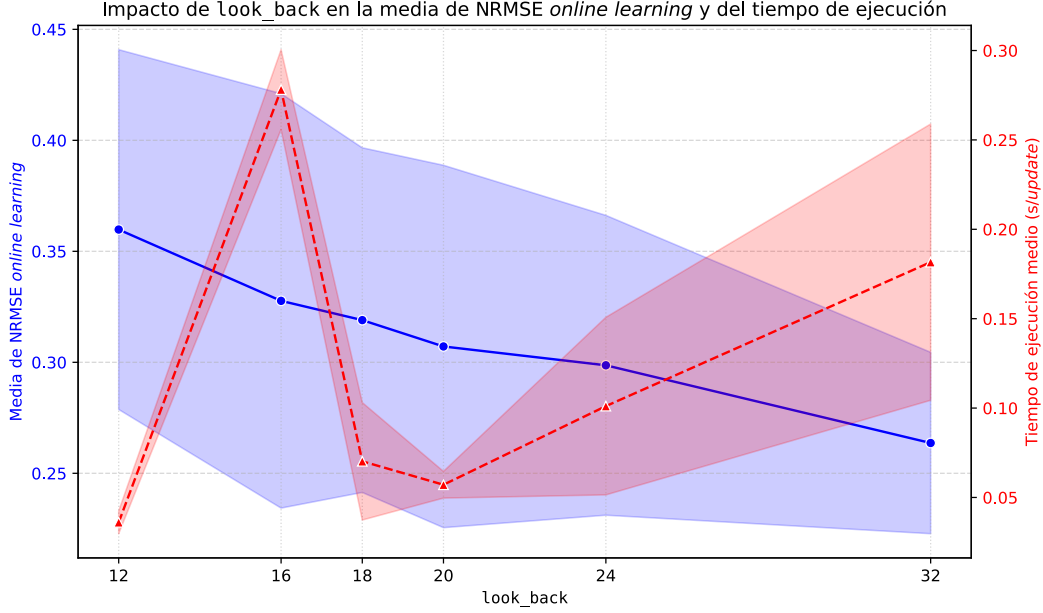


Figura 4: Influencia del hiperparámetro `look_back` en la media del NRMSE y en el tiempo de ejecución medio en modelos con *online learning*. Elaboración propia.

Si nos fijamos en el impacto de `look_back` en la media del NRMSE en modelos con entrenamiento estático es completamente al revés que en modelos con *online learning*. Su coeficiente de Spearman positivo ($r_s = 0,3183$) indica el potencial de *overfitting* latente en el hiperparámetro. Cuando la red neuronal procesa secuencias demasiado largas aprende demasiado bien los datos de entrenamiento, como así lo indica su coeficiente de Spearman ($r_s = -0,2321$) y no generaliza correctamente.

Encontrar valores para incluir en el *grid search* del hiperparámetro `batch_size` no es tarea sencilla. Continuando con el trabajo del Dr. D. Juan Pablo Casaseca de la Higuera [10], en este trabajo se han probado valores en torno a *batches* de 25 secuencias temporales. Al igual que con el hiperparámetro `look_back`, su coeficiente de Spearman respecto al tiempo de ejecución es positivo ($r_s = 0,1711$) por lo que *batches* de muchas secuencias temporales no son viables.

Cada *batch* contiene una cantidad limitada de secuencias temporales dispuestas de una forma peculiar. Cada secuencia está ordenada de tal forma que crean una ventana deslizante en los datos, por ello en el Capítulo ?? se ha denominado al hiperparámetro `look_back` como tal. De esta forma, al escoger valores en torno a 25 para `batch_size`, la red neuronal procesa en cada *batch* varias series temporales parcialmente superpuestas que abarcan aproximadamente 48 horas desde la primera muestra de la primera secuencia hasta la última muestra de la última secuencia. Esto permite a la red neuronal analizar el patrón subyacente de dos días naturales.

Esto es especialmente importante en los modelos con *online learning* puesto que, tras el entrenamiento, **batch_size** dicta la cantidad de información a la que la red neuronal tiene acceso para ajustar sus pesos. Cuanta más información tiene la red, más precisos son sus ajustes ($r_s = -0,1993$). Sin embargo, en el entrenamiento ($r_s = -0,0055$) y test sin *online learning* ($r_s = -0,0087$) el impacto es despreciable puesto que la red neuronal tiene acceso a todo el dataset de entrenamiento.

6.2.3. updates y threshold

Los hiperparámetros **updates** y **threshold** sólo tienen efecto en los modelos con *online learning*. Por ello, aunque algunos de sus coeficientes de Spearman con las métricas de modelos *offline* sean altos, es una coincidencia llamada relación espuria.

Updates indica cuántas veces se procesa el mismo *batch* en *online learning*. Cada vez que se procesa un *batch* se realiza el proceso de *backpropagation* una vez. Por lo tanto, cuantas más veces se realice, mayor precisión tendrá el ajuste de pesos y mayor será la carga computacional y el tiempo de ejecución.

Los coeficientes de Spearman de **updates** respecto a la media de RNMSE con *online learning* ($r_s = -0,3473$) y al tiempo de ejecución ($r_s = 0,5482$) indican que es uno de los hiperparámetros más importantes aunque también uno de los más peligrosos. Al igual que **look_back**, cuantas más veces se ajuste la red neuronal al *batch* actual, mayor será su precisión pero mayor será su coste computacional. Además, el coste computacional también está agravado por la arquitectura de la red neuronal, a mayor cantidad de capas se necesitan más cálculos para realizar *backpropagation*.

Como se observa en la Figura 5, los mejores resultados se encuentran a mayor número de **updates** pero la carga computacional es mayor. Además, se ha de tener en cuenta otros factores que agravan este problema como el tamaño de la cuadrícula, teniendo también un coeficiente de Spearman respecto al tiempo de ejecución bastante alto ($r_s = 0,3127$). Esto no suele ser un problema en modelos desplegados en entornos reales puesto que los procesadores suelen tener más capacidad de procesamiento y no tienen las restricciones impuestas en el Capítulo ?? para la reproducibilidad de los resultados.

En cuanto al hiperparámetro **threshold**, indica si el *batch* actual ha de seguir siendo procesado. No es un hiperparámetro que haya generado un gran impacto puesto que su coeficiente de Spearman es despreciable ($r_s = 0,0362$). Como las actualizaciones de los pesos se han realizado siguiendo el modelo propuesto por el Dr. D. Juan Pablo Casaseca de la Higuera [10], las actualizaciones de pesos provocadas por errores pequeños son menos potentes que las realizadas por los errores mayores. Por lo tanto, aunque el hiperparámetro **threshold** elimine las actualizaciones provocadas por errores pequeños, no son actualizaciones influyentes. Sin embargo, al eliminarlas se libera una gran cantidad de carga computacional, como se puede comprobar a través de su coeficiente de Spearman correspondiente ($r_s = -0,2177$).

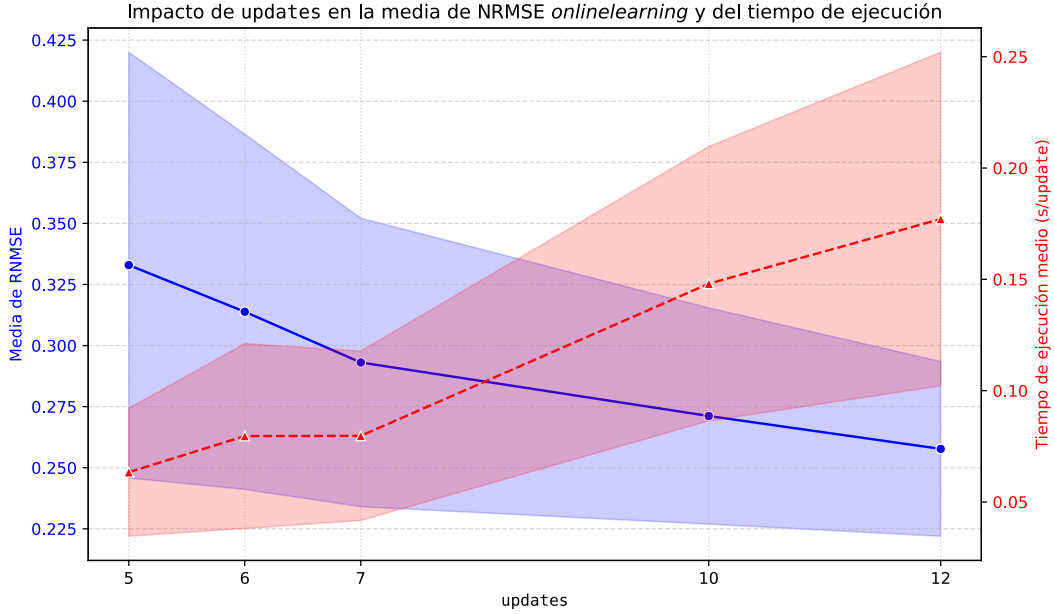


Figura 5: Influencia del hiperparámetro `updates` en la media del NRMSE y en el tiempo de ejecución medio en modelos con *online learning*. Elaboración propia.

6.2.4. Número de filtros de las capas convolucionales

El número de filtros indica la capacidad de las capas convolucionales para analizar patrones geográficos en la cuadrícula analizada. Sin embargo, un gran número de filtros puede provocar *overfitting*. Cuando hay demasiados filtros la red comienza a identificar el ruido presente en los datos como un patrón legítimo.

En cuanto al tiempo de ejecución, el resultado es contraintuitivo ($r_s < 0$). La respuesta está en la librería `Tensorflow` y la forma en que realiza los ajustes en los modelos con *online learning*. En cada modelo, la primera iteración de *online learning* es extremadamente lenta puesto que `Tensorflow` realiza un mapa de la red neuronal y lo convierte en una serie de funciones altamente optimizadas con el algoritmo *Tracing Graph*. En las siguientes iteraciones ya tiene el mapa hecho y lo ejecuta velozmente. Como se ha argumentado antes, las redes con más filtros son más propensas al *overfitting*, lo que aumenta el error NRMSE en las predicciones. Errores mayores tienen más probabilidad de superar el valor del hiperparámetro `threshold`, lo que genera más iteraciones de *online learning*. Todas esas iteraciones adicionales son extremadamente rápidas en comparación con la primera iteración, lo que reduce la media del tiempo de ejecución [14, 25].

6.3. Mejor modelo

El grueso de las medias de NRMSE se encuentra entre 0.20 y 0.50. En sí no son valores satisfactorios puesto que el objetivo se encuentra en un $\text{NRMSE} < 0.1778$. Sin embargo, aunque en media no se alcance el objetivo, es probable que, escogiendo los modelos con mejor media de NRMSE, algunas celdas superen el objetivo.

Los modelos sin capas convolucionales se quedan atrás en su rendimiento medio respecto a los modelos con capas convolucionales como se observa en la Figura 2. La cuadrícula de 49 celdas también es la que resulta más sencilla de procesar para las redes neuronales (*véase Figura 1*).

El modelo con menor media de NRMSE en *online learning* a lo largo de todas sus celdas es un modelo con una capa convolucional y otra capa de neuronas LSTM con la siguiente configuración:

Hiperparámetro	Valor
Tamaño cuadrícula	49
Capas CNN	1
Tamaño kernel	2
look_back	24
batch_size	25
updates	12
threshold	0.1
Filtros capa 1	16
Filtros capa 2	—
Filtros capa 3	—
Métricas de Rendimiento	
NRMSE entrenamiento	0.1221
NRMSE test <i>offline</i>	0.3487
NRMSE test <i>online</i>	0.2048
Tiempo de ejecución medio (s)	0.0768

Tabla 2: Características del mejor modelo en *online learning* y sus métricas de rendimiento como media del NRMS de sus celdas. Elaboración propia.

Podemos observar en la Figura 6 cómo se reparte la media NRMSE=0,2048 entre las 49 celdas. Cada celda tiene un dos números, el superior indicando el identificador de la celda en el dataset Milán y el inferior indicando el NRMSE. No hay nada que indique una distribución concreta ni una relación clara entre la posición de las celdas y su NRMSE. Destacan como celdas con menor NRMSE las celdas 5263, 5163 y 4962 y las celdas 5357, 5058 y 4959 como celdas con peor NRMSE.

NRMSE por celda del mejor modelo *online learning*

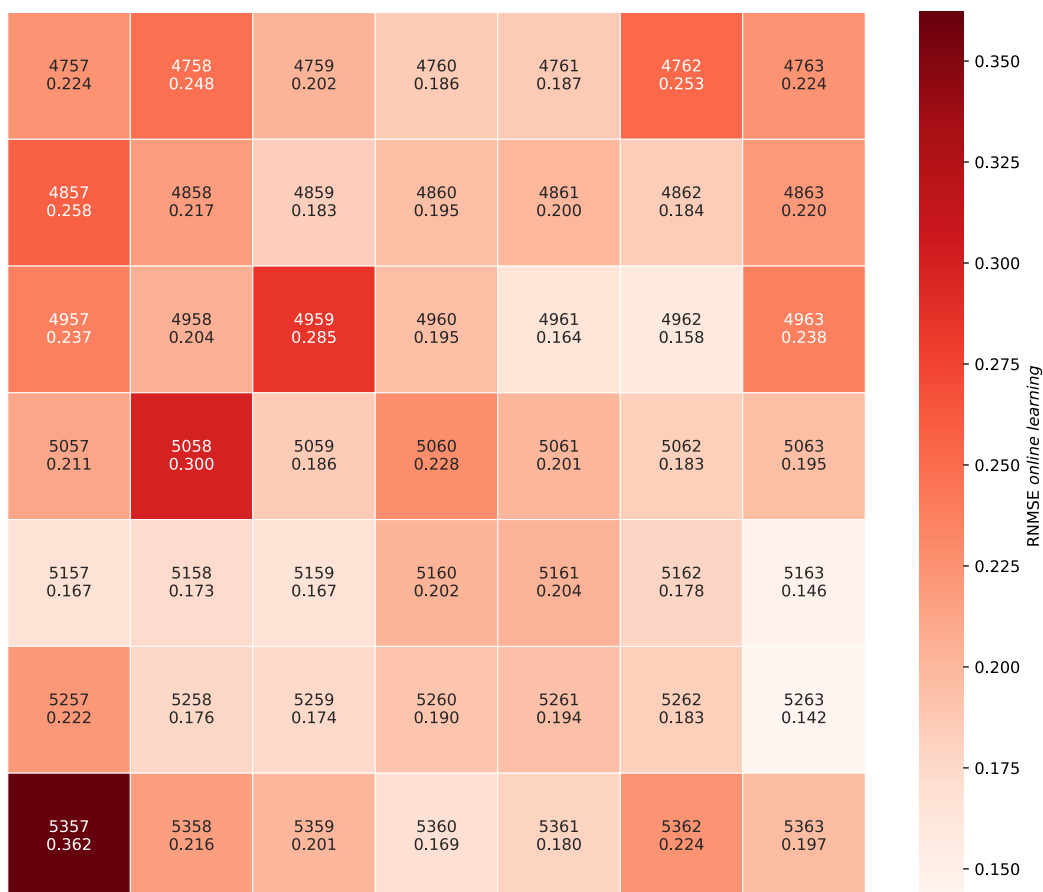


Figura 6: NRMSE por celda del mejor modelo con *online learning*. Elaboración propia.

Al comparar el diagrama de calor de la Figura 6 con los diagramas de calor obtenidos en los mejores modelos de *online learning* de otras arquitecturas, se aprecia que todas las arquitecturas tienen más o menos la misma distribución de errores. Las mejores celdas siempre son las celdas 5263, 5163 y 4962 y las peores celdas siempre son las celdas 5357, 5058 y 4959, como se observa en la Figura 7.

NRMSE por celda de los mejores modelos *online learning*

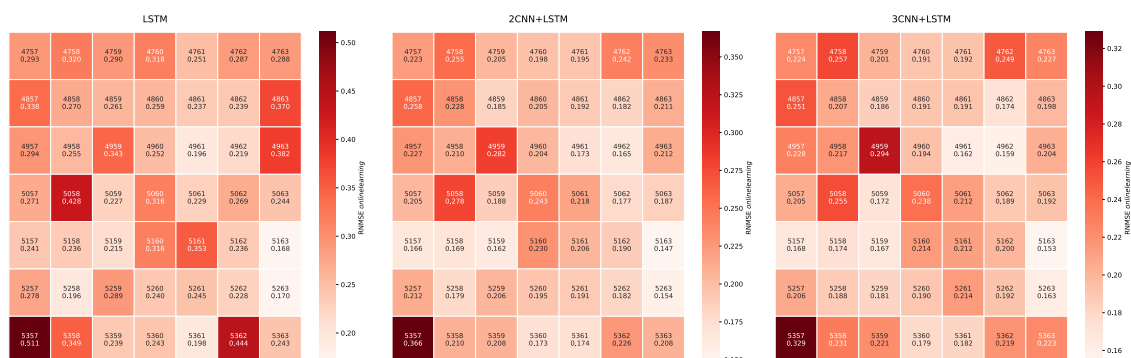


Figura 7: NRMSE por celda de los mejores modelos con *online learning* de cada arquitectura (LSTM, 2CNN+LSTM y 3CNN+LSTM). Elaboración propia.

Para obtener más detalles es necesario observar las predicciones realizadas en cada celda. No es necesario ni eficiente analizar todas las celdas por lo que sólo se escrutarán las 3 mejores y las 3 peores. Como el modelo tiene una arquitectura con una capa convolucional con un kernel 3x3 y una capa recurrente LSTM, es interesante comparar las celdas analizadas con las celdas contiguas para intentar estimar qué datos han tenido en cuenta las neuronas convolucionales.

Respecto a las mejores celdas, las celdas 5263 y 5163 se influyen mutuamente al ser contiguas. Observando los datos de test, ambas tienen un marcado patrón periódico diario y semanal, en el que los primeros cinco días tienen más tráfico que los dos días siguientes. Mirando las celdas contiguas a las mismas, las celdas 5063, 5062 y 5363 presentan el mismo patrón. Las celdas 5162 y 5262 presentan un patrón en el que los siete días de la semana son prácticamente idénticos con mínimos más bajos que las otras celdas. Observando las predicciones sin *online learning*, parece que éstas últimas han influido en las predicciones de las celdas 5263 y 5163 y la red neuronal ha infravalorado el tráfico de red futuro.

Respecto a las peores celdas, las celdas 4959 y 5357 no son fáciles de predecir por ninguna red neuronal. Los datos de test de la celda 4959 contiene mucho ruido y cambios bruscos y rápidos alrededor de los mínimos de tráfico y la celda 5057 presenta un cambio radical en el patrón subyacente de los datos. Los modelos sin *online learning* no son capaces de soportar un cambio tan grande. Aun así, parece que aunque la celda 5357 tiene un patrón más o menos uniforme antes del cambio, la influencia del patrón en forma de 5 y 2 días de las celdas adyacentes se puede ver en las predicciones sin *online learning*.

En general, la mayor contribución al NRMSE es provocado por anomalías en los patrones, como por ejemplo los máximos abruptos de la celda 5058. Sin embargo, es visible en todas las celdas cómo los modelos con *online learning* son muy superiores a aquellos sin *online learning*. Esto se observa muy bien en las celdas con menor precisión, por ejemplo, el modelo con *online learning* ha podido adaptarse rápidamente los cambios radicales de las celdas 5357 y 5362 en cuestión de 2 o 3 días. Por otra parte, el modelo sin *online learning* son completamente incapaces de adaptarse.

7. Conclusiones y líneas futuras

Este trabajo ha plasmado la investigación realizada en torno a las redes neuronales compuestas por capas convolucionales y recurrentes aplicadas a la predicción del tráfico en red con *online learning*. Se han analizado varias arquitecturas de redes neuronales con una capa de neuronas LSTM precedida por una cantidad variable entre ninguna y tres capas CNN. Se ha realizado un extenso proceso de validación basado en la estrategia *grid search* sobre el dataset Milán y se ha obtenido el modelo con mayor precisión utilizando el NRMSE como métrica de rendimiento.

Como regla general, los modelos con *online learning* son ampliamente superiores a aquellos sin *online learning*. Los cambios en el patrón de los datos pasan completamente desapercibidos para los modelos sin *online learning*. En cambio, los modelos con *online learning* presentan una capacidad de adaptación extraordinaria.

La implementación de capas convolucionales al modelo de predicción de series temporales basado únicamente en capas de neuronas LSTM ha dado resultado, reduciendo los errores cometidos en las predicciones. Respecto al modelo original cedido por el Dr. D. Juan Pablo Casaseca de la Higuera, se ha conseguido reducir el error de predicción desde $\text{NRMSE}=0,1778$ hasta $\text{NRMSE}=0,142$.

Por último, se ha alcanzado la optimización del modelo original a través de funciones altamente eficientes proporcionadas por la librería **Tensorflow**. Gracias a ello se ha conseguido reducir considerablemente el tiempo de ejecución requerido por los modelos para los procesos de *backpropagation* y *online learning*.

Para próximos trabajos se plantea la posibilidad de implementar los modelos descritos en este trabajo como redes de grafos orientadas a la predicción de series temporales, más concretamente a la predicción de tráfico de red.

Referencias

- [1] G. Bell, “Bell’s law for the birth and death of computer classes,” *Communications of the ACM*, vol. 51, no. 1, pp. 86–94, Jan 2008. [Online]. Available: <https://doi.org/10.1145/1327452.1327453>
- [2] A. Azzouni and G. Pujolle, “A long short-term memory recurrent neural network framework for network traffic matrix prediction,” *arXiv preprint arXiv:1705.05690*, Jun 2017. [Online]. Available: <http://arxiv.org/abs/1705.05690>
- [3] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin, *Learning from Data: A Short Course*. United States: AMLBook, 2012.
- [4] N. Bui, M. Cesana, S. A. Hosseini, Q. Liao, I. Malanchini, and J. Widmer, “A survey of anticipatory mobile networking: context-based classification, prediction methodologies, and optimization techniques,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1790–1821, 2017.
- [5] T. Guo, Z. Xu, X. Yao, H. Chen, K. Aberer, and K. Funaya, “Robust online time series prediction with recurrent neural networks,” in *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016, pp. 816–825.
- [6] Y. Duan, G. Fu, N. Zhou, X. Sun, N. C. Narendra, and B. Hu, “Everything as a service (xaas) on the cloud: Origins, current and future trends,” pp. 621–628, 2015.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under Concept Drift: A Review,” *IEEE Transactions on Knowledge & Data Engineering*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TKDE.2018.2876857>
- [8] G. Widmer and M. Kubat, “Learning in the presence of concept drift and hidden contexts,” *Machine Learning*, vol. 23, no. 1, pp. 69–101, 1996. [Online]. Available: <https://doi.org/10.1007/BF00116900>
- [9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, Mar 2014. [Online]. Available: <https://dl.acm.org/doi/epdf/10.1145/2523813>
- [10] P. Casaseca-de-la Higuera, J. M. Aguiar, P. Amado-Caballero, A. J. Morgado, J. d. S. Moreira, C. Luo, and X. Wang, “Online active learning for LSTM-based network traffic prediction,” 2023, iEEE (Unpublished Draft / Preprint).
- [11] A. Román Antolín, “Predicción de tráfico de red usando arquitecturas neuronales avanzadas,” Trabajo Fin de Grado, Universidad de Valladolid, Valladolid, España, Sep 2025.
- [12] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: <http://jmlr.org/papers/v13/bergstra12a.html>

- [13] D. H. Wolpert, “The lack of a priori distinctions between learning algorithms,” *Neural Computation*, vol. 8, no. 7, pp. 1341–1390, 10 1996. [Online]. Available: <https://doi.org/10.1162/neco.1996.8.7.1341>
- [14] F. Chollet, *Deep learning with Python*, 2nd ed. Madrid: Anaya Multimedia, 2020.
- [15] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems*, 3rd ed. Beijing: O’Reilly, 2022.
- [16] L. N. Smith, “A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay,” *CoRR*, vol. abs/1803.09820, 2018. [Online]. Available: <http://arxiv.org/abs/1803.09820>
- [17] M. González and I. M. del Puerto, *Series Temporales*, ser. Colección manuales uex. Cáceres, España: Universidad de Extremadura, Servicio de Publicaciones, 2009, vol. 60.
- [18] J. C. García Díaz, *Predicción en el dominio del tiempo: análisis de series temporales para ingenieros*, ser. Manual de referencia. Valencia: Editorial de la Universidad Politécnica de Valencia, 2016.
- [19] P. Hoong, “Bittorrent network traffic forecasting with arma,” *International Journal of Computer Networks and Communications*, vol. 4, pp. 143–156, 07 2012.
- [20] H. L. S., “A study in the analysis of stationary time series. by herman wold. [pp. 214 + viii. almqvist and wiksell boktryckeri-a.-b., uppsala. 1938. price kr. 6.],” *Journal of the Institute of Actuaries*, vol. 70, no. 1, p. 113–115, 1939.
- [21] K. H. Poo, I. K. T. Tan, and K. H. Ng, “Computing autoregressive moving average (ARMA) with CRONOS: A case study of Bursa Malaysia,” in *Proc. 2nd Int. Conf. Eng. ICT (ICEI)*, Melaka, Malaysia, Feb 2010, pp. 516–520.
- [22] M. Papadopouli, H. Shen, E. Raftopoulos, M. Ploumidis, and F. Hernandez-Campos, “Short-term traffic forecasting in a campus-wide wireless network,” in *2005 IEEE 16th International Symposium on Personal, Indoor and Mobile Radio Communications*, vol. 3, 2005, pp. 1446–1452.
- [23] S. Jung, C. Kim, and Y. Chung, “A prediction method of network traffic using time series models,” vol. 3982, 11 2006, pp. 234–243.
- [24] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 1970. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/9781118619193>
- [25] TensorFlow Developers, “Better performance with tf.function,” 8 2024, [Accedido: May. 20, 2025]. [Online]. Available: <https://www.tensorflow.org/guide/function>